Diamond Framework:

Reputation-Weighted Validator Selection via Verifiable Non-Transferable Reputation

Ilich Blanco*

October 2026

Abstract

The Diamond Framework addresses the structural tension in Proof-of-Stake between security and accessibility. High collateral requirements deter honest but capital-constrained validators; low requirements expose the network to Sybil attacks. The framework replaces a fraction of the financial stake with verifiable, non-transferable reputation.

The paper makes four contributions: a cryptoeconomic protocol in which reputation lowers the financial stake that admission requires — a reputation-weighted access cost, not collateral at risk; a numerical verification suite that pairs every claim backed by a reproducible script with that script and states the status of the rest; a falsification log documenting twenty-two claims that were made, tested, and corrected during development; and a set of design decisions with explicit boundaries, stating where the protocol cannot deliver a guarantee.

The protocol is verified numerically for the reputation dynamics, the tensor representation, the collusion detection, the payment rule, the data layer, and the bootstrap transition. The zero-knowledge threshold proof is analysed: the original circuit is shown unsound, and the delivered V3/Groth16 circuit is measured in the repository (94,438 constraints, 3.9 s proving, 128-byte proof); the on-chain verifier is not deployed with a ceremony key, although a development-key verifier runs on the Fuji testnet; EVM precompile costs are measured locally with Foundry. Two benchmarks are deferred: the recursive SNARK proving time (not used by the adopted design) and the Teleporter latency, both requiring infrastructure not available in this paper.

Contents

1 Introduction **4**
- 1.1 The Validator Trust Gap . . . 4
- 1.2 Reputation as a Weighted Access Cost . . . 4
- 1.3 Contributions . . . 5
- 1.4 Paper Structure . . . 5
- 1.5 Reproducibility . . . 6

2 Reputation Dynamics **6**
- 2.1 The CIR Process . . . 6
- 2.2 Domain Validity Under Jumps . . . 6
- 2.3 The Whitewashing Bound . . . 7
- 2.4 Stationary Distribution . . . 8
- 2.5 Integro-Differential Formulation . . . 9
- 2.6 Summary of the Section . . . 9

*Andromeda Computer, ilichblanco@andromedacomputer.net

3 Tensor Compression **10**
3.1 Tensor Representation of Reputation . . . 10
3.2 Tucker Decomposition . . . 10
3.3 Reconstruction Error . . . 10
3.4 Error Versus Rank . . . 11
3.5 Tucker Is Not in the Security Path . . . 11
3.6 Summary of the Section . . . 12

4 Zero-Knowledge Threshold Proof **12**
4.1 Circuit Specification . . . 12
4.2 Soundness Failure . . . 13
4.3 Repair Options . . . 13
4.4 Direct Verification and the Recursive Option . . . 14
4.5 Succinctness and the Cost of Ingesting the Tensor . . . 15
4.6 Estimates and Their Provenance . . . 15
4.7 Summary of the Section . . . 15

5 Collusion Detection **16**
5.1 The Collaboration Graph . . . 16
5.2 The Normalization Problem . . . 16
5.3 The Modularity Matrix . . . 18
5.4 Null Distribution and Calibration . . . 18
5.5 Detection Power . . . 19
5.6 The Detection Boundary . . . 20
5.7 Limitations . . . 20
5.8 Summary of the Section . . . 20

6 Payment Rule **21**
6.1 The Revelation Problem . . . 21
6.2 Verified Reputation . . . 21
6.3 Private Cost of Capital . . . 21
6.4 The Uniform Posted Price . . . 22
6.5 Individual Rationality . . . 22
6.6 Sensitivity . . . 23
6.7 Incentive Compatibility . . . 23
6.8 Summary of the Section . . . 23

7 Data Layer **24**
7.1 Ingestion of Consensus Events . . . 24
7.2 Transport and Freshness . . . 24
7.3 The Fraud Proof Mechanism . . . 24
7.4 Numerical Verification . . . 25
7.5 Slashing and Integrity . . . 26
7.6 Freshness as Second Line of Defense . . . 27
7.7 Summary of the Section . . . 28

8 Bootstrap Dynamics **28**
8.1 The Genesis Problem . . . 28
8.2 Reputation Models . . . 28
8.3 Bootstrap Simulation . . . 29
8.4 Results . . . 29
8.5 Comparison of Policies . . . 30
8.6 Recommendation . . . 30
8.7 Summary of the Section . . . 30

9 Evaluation **31**
9.1 Scope . . . 31
9.2 Reproducibility . . . 31
9.3 On-Chain Gas Costs . . . 31
9.4 Stability of Conclusions . . . 32
9.5 Deferred Benchmarks . . . 33
9.6 Summary of the Section . . . 34

10 Limitations **34**
10.1 Scope . . . 34
10.2 Adversary Model . . . 34
10.3 Collusion Detection Boundary . . . 36
10.4 Collusion by Bipartite Polarization . . . 37
10.5 Reputation Model Selection . . . 37
10.6 Community Structure and Tucker Compression . . . 37
10.7 Calibration on Synthetic Graphs . . . 37
10.8 Bootstrap Assumptions . . . 37
10.9 What the Protocol Does Not Claim . . . 38
10.10 Summary of the Section . . . 38

11 Conclusion **39**
11.1 Summary . . . 39
11.2 Methodology . . . 40
11.3 Contributions . . . 40
11.4 Open Problems . . . 40
11.5 Closing Remark . . . 41
11.6 Summary of the Section . . . 41

A Reproduction Protocol **41**

1 Introduction

1.1 The Validator Trust Gap

Proof-of-stake networks secure their consensus by requiring validators to lock financial collateral. Slashing conditions make misbehavior costly, and the resulting cryptoeconomic security guarantees are well formalized. This mechanism has proven effective. It has also produced a structural tension.

High stake requirements deter honest but capital-constrained participants, concentrating validation power in the hands of large token holders. Low requirements expose the network to Sybil attacks and near-zero-cost misbehavior. The tension is not a design flaw of any particular protocol; it is a consequence of using a single instrument—financial capital—to serve two purposes simultaneously: deterrence and admission.

In proof-of-work networks, physical capacity constraints (hardware, energy, cooling) act as a natural brake on centralization. A miner with a technological advantage cannot expand indefinitely without facing diminishing returns on capacity. In proof-of-stake networks the collateral is purely financial and does not face an equivalent physical brake. The resulting concentration dynamics can be self-reinforcing.

1.2 Reputation as a Weighted Access Cost

The Diamond Framework introduces a second instrument to the admission decision: verifiable, non-transferable reputation. A validator with a long history of contributions faces a large opportunity cost if it misbehaves, because destruction of its reputation eliminates future income streams. This opportunity cost lowers the financial barrier for honest participants by discounting the stake the rule requires: a reputation-weighted access cost, not collateral that a slash can seize. Whether this preserves the network's Cost of Attack is an open question, taken up in Section 6 and recorded in Section 10.

Non-transferability applies to the reputation state, not to the validator's identity. The protocol assumes an identity layer that makes a validator identity persistent and costly to recreate; it does not provide that layer. Without it, selling a signing key would transfer the reputation-weighted income stream even though the reputation token itself cannot be transferred. A secondary market for identities or keys is outside the scope of the model and is declared as a limitation in Section 10.

Making this intuition rigorous requires solving several problems:

1. **Stochastic modeling.** Reputation evolves continuously and is subject to abrupt downward shocks. The dynamics must remain in their positive domain under all admissible shocks.

2. **Verifiability.** Reputation claims must be cryptographically verifiable on-chain and without relying on a centralized oracle. The tensor T is public (Section 7), so the proof provides succinctness rather than privacy.

3. **Collusion resistance.** Validators may coordinate to inflate each other's reputation. The detection mechanism must be robust to heavy-tailed interaction graphs and must have a stated boundary below which detection is information-theoretically impossible.

4. **Incentive compatibility.** The mechanism must ensure validators report their true reputation, or—if reputation is publicly verified—must make the reporting question vacuous.

5. **Data integrity.** The reputation signal is derived from consensus events on source chains. The transport of these events must be verifiable, and fraud must be detectable and deterred.

The Diamond Framework addresses all five.

1.3 Contributions

The paper makes four contributions.

A cryptoeconomic protocol. We present a validator admission rule in which reputation lowers the financial stake that admission requires. The rule is $\mathrm{CoA}(i) = S_f(i) + C_R(T(i)) \geq \Theta(t)$, where $C_R(T)$ is a reputation-weighted access cost computed from the validator's verified entry $T(i)$: a credit against the financial stake the rule requires, and not collateral that the protocol can seize. This is a per-validator admission condition, not a network-level cost-of-attack bound; Section 6 states the distinction and Section 10 records what remains open. The protocol includes the reputation dynamics, the collusion detection, the payment rule, the data layer, and the bootstrap simulation.

A numerical verification suite. Every claim backed by a reproducible script is paired with it. The scripts are self-contained, depend only on NumPy and SciPy, and declare their seeds and sample sizes. The suite covers stochastic simulation, spectral calibration, game-theoretic equilibrium, and EVM gas benchmarks. Claims that are estimates, design choices, deferred benchmarks, or conjectures state their status and, where applicable, the source of the estimate.

A falsification log. During development, twenty-two quantitative claims were made, tested, and either confirmed or corrected. The corrections include a fabricated simulation figure, a mismeasurement of a gas cost, a factor of two in an integral, a rounding rule that shifted a phase transition, and an unverified projection of a multidimensional type. The log is part of the paper's record, not a supplement to it.

Design decisions with explicit boundaries. Where the protocol cannot deliver a guarantee—for adversaries that are not rational, for collusion below the Kesten–Stigum boundary, for real reputation data that is not available—the paper states the boundary and does not claim the guarantee. The limitations section separates methodological limits (the numerical methods could be improved) from design limits (the protocol does not solve these problems even with perfect methods).

Positioning. The protocol sits between two literatures and is not a variant of either. EigenTrust-style systems aggregate peer feedback into a reputation score but do not price admission with it [11]. Delegated proof-of-stake and stake delegation move voting weight to delegates or pools, but the delegated weight is transferable and is not a reputation state derived from consensus events [13, 9]. In this paper reputation is derived from finalized consensus events, is bound to a validator identity, and enters the admission rule only as a discount on the financial stake the rule requires; it is not a transferable token, a delegated voting weight, or a weight in consensus. The identity and delegation layers that this positioning assumes are stated as limitations in Section 10.

1.4 Paper Structure

Section 2 presents the reputation dynamics: a CIR diffusion with multiplicative downward jumps, the positivity proof, and the exact mean first-passage time.

Section 3 introduces the tensor representation and the Tucker compression, and shows that the compression fails for block structure while remaining useful for low-rank tensors.

Section 4 specifies the zero-knowledge threshold proof. The original circuit is unsound; the delivered repair is a V3 Poseidon-sponge circuit with Groth16 verification, measured in the repository. The EVM precompile costs are measured locally.

Section 5 develops the collusion detection. The modularity matrix $B = W - dd^\top/(2m)$ replaces the normalized adjacency matrix A_{sym}. The null quantile is calibrated by Monte Carlo, and the detection boundary is characterized.

Section 6 specifies the payment rule: a saturated uniform posted price. Incentive Compatibility is tautological because reputation is verified on-chain and the allocation does not depend on any private type.

Section 7 describes the data layer: the ingestion of consensus events, the transport via state inclusion proofs, and the fraud proof mechanism with its audit equilibrium.

Section 8 presents the bootstrap simulation under the net_gain reputation model, with the comparison of three policies for the transition from pure proof-of-stake to hybrid.

Section 9 evaluates reproducibility, on-chain gas costs, and the stability of the conclusions under parameter variation.

Section 10 states the limitations of the design and of the numerical verification, separated by category.

Section 11 concludes.

1.5 Reproducibility

The paper is accompanied by a public repository containing:

- The verification scripts (`verification/`).
- The EVM benchmarks (`integrations/avalanche/benchmarks/`).
- The falsification log (`docs/audit/numerical-verification.md`).
- The paper source (`docs/paper/main.tex`).

Every claim backed by a script, a benchmark, or a log entry can be traced to that source. The commands to reproduce the claims are listed in `verification/README.md`. Running the full suite takes approximately 67 minutes on a single core, dominated by the heavy-tailed recalibrations (`heavy_tail_n5000.py`, ∼15 min; `heavy_tail_scaled_clique.py`, ∼24 min) and the RMT calibration (`recalibrate_top.py`, ∼10 min).

2 Reputation Dynamics

2.1 The CIR Process

Validator reputation is modeled as a mean-reverting stochastic process with abrupt downward shocks. In its continuous form, the process follows the Cox–Ingersoll–Ross (CIR) diffusion:

$$dR(t) = \alpha(\bar{R} - R(t))\,dt + \sigma_0\sqrt{R(t)}\,dW(t) \tag{2.1}$$

with $\alpha > 0$ the mean-reversion speed, $\bar{R} > 0$ the long-run mean, and $\sigma_0 > 0$ the diffusion coefficient. The process is strictly positive under the Feller condition $2\alpha\bar{R} \geq \sigma_0^2$, which we assume throughout.

Slashing events are modeled as multiplicative downward jumps:

$$R(t) \mapsto R(t^-) \cdot y, \quad y \in (0, 1] \tag{2.2}$$

where y is drawn from a Lévy measure ν supported on $(0, 1]$. The choice of multiplicative rather than additive jumps is not cosmetic; it is required for the process to remain in its domain.

2.2 Domain Validity Under Jumps

The original formulation of the Diamond Framework used additive jumps of the form:

$$dR(t) = \alpha(\bar{R} - R(t))\,dt + \sigma_0\sqrt{R(t)}\,dW(t) + \int J(z)\tilde{N}(dt, dz) \tag{2.3}$$

with $J(z) < 0$. This specification is not well posed. Additive negative jumps can push R below zero regardless of the Feller condition, because the Feller condition constrains the diffusion, not the jump component.

We quantify this failure by Monte Carlo simulation. With parameters $\alpha = 0.5$, $\bar{R} = 1.0$, $\sigma_0 = 0.8$ (Feller satisfied: $2\alpha\bar{R} = 1.0 \geq \sigma_0^2 = 0.64$), jump intensity $\lambda = 0.8$, mean jump magnitude 0.15, $R(0) = 0.3$, and $T = 5$, the additive-jump specification produces (Table 2.1):

Table 2.1: Fraction of simulated trajectories that leave the positive domain, additive versus multiplicative jump specifications.

Specification	Fraction with $R(t) < 0$
Additive jumps	40.58%
Multiplicative jumps	0.0000%

over 100,000 simulated paths with $dt = 0.001$ [`verification/stochastics/sim_positivity.py`].

The failure rate of 40.58% is not a tail event. With the specified parameters, two in five trajectories violate the domain within five time units. The multiplicative specification resolves this by construction: a jump of the form $R \mapsto R \cdot y$ with $y \in (0, 1]$ preserves positivity pathwise, and the CIR component remains positive under Feller.

2.3 The Whitewashing Bound

For a validator whose reputation has been reduced to near zero, the expected time to recover to a target level R_0 is governed by the first-passage time of the CIR diffusion. Under the scale function $s'(x) = x^{-\kappa} \exp(\beta x)$ with $\kappa = 2\alpha\bar{R}/\sigma_0^2$ and $\beta = 2\alpha/\sigma_0^2$, the classical expression for the expected first-passage time is:

$$\mathbb{E}[\tau_{R_0}] = 2 \int_0^{R_0} s'(z) M(z)\, dz \tag{2.4}$$

where $M(z) = \int_0^z m'(y)\, dy$ and $m'(y) = 1/(\sigma^2(y) s'(y))$ is the speed measure density.

We evaluate (2.4) by numerical quadrature in the well-conditioned form

$$\mathbb{E}[\tau_{R_0}] = 2 \int_0^{R_0} g(y)\, dy, \quad g(y) = \frac{1}{\sigma_0^2} \int_1^{R_0/y} s^{-\kappa} \exp(\beta y (s-1))\, ds \tag{2.5}$$

which avoids the catastrophic cancellation that arises from computing $s(R_0) - s(y)$ directly when $\kappa > 1$ [`verification/stochastics/mfpt_bound_explicit.py`].

For the two parameter regimes of interest (Table 2.2):

Table 2.2: Exact mean first-passage time against the bound with $C^* = 1$ and the implicit C^*, for the two parameter regimes.

Parameters	Exact MFPT	Bound with $C^* = 1$	Implicit C^*
$\alpha = 0.5$, $\sigma_0 = 0.8$	11.2203	19.965	0.562
$\alpha = 2.0$, $\sigma_0 = 0.5$	443.7374	1.415×10^{10}	3.14×10^{-8}

The values above are the mean first-passage time of the CIR diffusion without jumps, i.e., the $\lambda = 0$ limit of the process defined in Section 2. The full process with multiplicative downward jumps has a larger MFPT: at $\lambda = 0.8$, the measured value is 19.38 in the first regime and 710.5 in the second, and the fraction of paths that reach R_0 within the horizon falls from 1.0 to 0.82 in the second [`verification/stochastics/mfpt_jump_diffusion.py`]. The whitewashing bound of this section is therefore a lower bound for the process as defined.

The analytical bound of the original formulation, $C^* \cdot R_0^{1-\kappa} \exp(\beta R_0)$, requires specifying C^*. With $C^* = 1$ the bound exceeds the exact MFPT in both regimes: by a factor of 1.78 in the first, and by 3.19×10^7 (7.5 orders of magnitude) in the second. $C^* = 1$ **is not a valid lower bound for the CIR first-passage time.** The implicit C^* values (0.562 and 3.14×10^{-8}) are the largest constants for which the bound holds.

We verified the numerical values by two independent methods. The first is direct Euler simulation of the CIR [`verification/stochastics/dt_convergence.py`]. The simulated MFPT converges monotonically to the exact value as $dt \to 0$ (Table 2.3):

Table 2.3: Mean first-passage time against the integration step, by direct simulation of the CIR component.

dt	MFPT ($\alpha = 0.5$, $\sigma_0 = 0.8$)	MFPT ($\alpha = 2.0$, $\sigma_0 = 0.5$)
0.02	12.3115 (+9.7%)	561.55 (+26.6%)
0.005	11.7708 (+4.9%)	503.71 (+13.5%)
0.001	11.3334 (+1.0%)	461.83 (+4.1%)
0.0005	11.2453 (+0.2%)	—

The Euler scheme with $\max(R, 0)$ truncation overestimates the MFPT because it delays arrival at the threshold. The bias is monotone in dt and consistent across regimes.

The second method is the censoring test [`verification/stochastics/censoring_test.py`], which characterizes the shape of the first-passage time distribution under censoring. In the first regime the process has no censoring within $T_{\max} = 300$: all 20,000 simulated paths reach R_0. In the second regime only 39.7% reach R_0 at $T_{\max} = 300$. The empirical tail is approximately geometric: the ratio $P_{\text{emp}}/P_{\text{geo}}$ decays from 0.9973 at $k = 2$ to 0.9858 at $k = 6$, with mean 0.9927 and std 0.0038 across five windows. The deviation from geometricity is small but systematic. Under the geometric assumption the unconditional MFPT would be $T_0/(1-p)$ with $p = P(\tau > T_0) = 0.9214$, giving 635.7. The exact MFPT is 443.7. The 43% gap quantifies the effect of the sub-geometric tail: the empirical survival decays faster than geometric, so the extrapolation overestimates the MFPT.

The two methods agree on the qualitative structure: the first-passage time is finite, grows with R_0, and is dominated by the ratio α/σ_0^2. The censoring test is a tail-shape diagnostic, not an independent estimate of the MFPT; its 43% overestimate is the numerical signature of the tail's sub-geometricity. No claim of superexponential growth is made.

2.4 Stationary Distribution

Under the Feller condition $2\alpha\bar{R} \geq \sigma_0^2$, the CIR process has a unique stationary distribution:

$$R_\infty \sim \text{Gamma}(k, \theta), \quad k = \frac{2\alpha\bar{R}}{\sigma_0^2}, \quad \theta = \frac{\sigma_0^2}{2\alpha} \tag{2.6}$$

with moments

$$\mathbb{E}[R_\infty] = k\theta = \bar{R}, \quad \text{Var}[R_\infty] = k\theta^2 = \frac{\sigma_0^2\bar{R}}{2\alpha} \tag{2.7}$$

We verify (2.6) and (2.7) by simulating the process from $R(0) = \bar{R}$ over $T = 50$, discarding the first 50% of samples as burn-in, and comparing empirical moments against the theoretical values [`verification/stochastics/stationary_stats.py`]. The results (Table 2.4):

Table 2.4: Theoretical and empirical stationary moments of the multiplicative specification.

Parameters	$\mathbb{E}[R_\infty]$ theory	empirical	Var theory	Var empirical
$\alpha = 0.5$, $\sigma_0 = 0.8$	1.000000	0.998296 (0.17%)	0.640000	0.637595 (0.38%)
$\alpha = 2.0$, $\sigma_0 = 0.5$	1.000000	1.000062 (0.006%)	0.062500	0.063097 (0.96%)

The relative differences are below 1% in all four quantities. The Z-score of the whitewashing target $R_0 = 2.2$ is 1.50σ in the first regime and 4.80σ in the second, consistent with empirical estimates (1.5050σ and 4.7770σ).

A two-sample Kolmogorov–Smirnov test against $\text{Gamma}(k, \theta)$ does not reject the stationary distribution at 95%: $D = 0.00354$, $p = 0.557$ (first regime); $D = 0.00479$, $p = 0.201$ (second regime). Because the simulated values are autocorrelated (50,000 paths $\times$ 2,500 steps), the KS p-value is indicative only. The moment comparison is the primary evidence.

2.5 Integro-Differential Formulation

For the Fokker–Planck density $p(r,t)$ of the reputation process, the forward equation is:

$$\frac{\partial p}{\partial t} = -\frac{\partial}{\partial r}[\mu(r)p] + \frac{1}{2}\frac{\partial^2}{\partial r^2}[\sigma^2(r)p] + \lambda \int_0^1 \left[\frac{p(r/y,t)}{y} - p(r,t)\right] \nu(dy) \tag{2.8}$$

with $\mu(r) = \alpha(\bar{R} - r)$, $\sigma^2(r) = \sigma_0^2 r$, and the jump term written for the multiplicative specification. The domain is $r \in [0, \infty)$. Under the Feller condition the process never reaches 0 from above, so no boundary condition is imposed there; outside it, 0 is absorbing.

The associated value function $C_R(r)$, the reputation-weighted access cost, satisfies the integro-differential equation:

$$\frac{\partial C_R}{\partial t} + \mu(r)\frac{\partial C_R}{\partial r} + \frac{1}{2}\sigma^2(r)\frac{\partial^2 C_R}{\partial r^2} + \lambda \int_0^1 [C_R(ry) - C_R(r)]\nu(dy) - rC_R = -\pi(r) \tag{2.9}$$

with boundary condition $C_R(0) = 0$, interpreted in the viscosity sense. Existence and uniqueness follow from standard arguments for integro-PDEs with polynomial growth of π [15]. The state r is the reputation level on the positive half-line; Section 6 evaluates C_R at the per-validator norm $r = T(i)$ that the admission circuit certifies (Section 4), and the reference configuration uses the closed form $C_R(r) = 0.8\,r/(1+r)$. The diffusion is solved on the half-line; the circuit's domain $r < 1$ bounds what admission can read, not what the dynamics can visit. The step from the diffusion to the committed scalar — which entry of the epoch each validator owns — is the attribution of Section 7, and it is not proved by the circuit.

The running reward implied by the closed form follows from (2.9). With $C_R'(r) = 0.8/(1+r)^2$, $C_R''(r) = -1.6/(1+r)^3$, and, for the reference jump $y = 0.85$ (mean magnitude 0.15), $C_R(yr) = 0.68\,r/(1+0.85r)$, the stationary operator ($\partial_t C_R = 0$) yields

$$\pi(r) = -\Big[\mu(r)\,\frac{0.8}{(1+r)^2} \;-\; 0.8\,\sigma^2(r)\,\frac{1}{(1+r)^3} \;+\; \lambda\Big(\frac{0.68\,r}{1+0.85r} - \frac{0.8\,r}{1+r}\Big) \;-\; \frac{0.8\,r^2}{1+r}\Big], \tag{2.10}$$

with $\mu(r) = \alpha(\bar{R} - r)$ and $\sigma^2(r) = \sigma_0^2 r$. [`verification/stochastics/cr_ide_solver.py`] evaluates (2.10) on a fixed grid: $\pi(r)$ is negative below an admission threshold $r_0 \approx 0.33$ (first regime) and ≈ 0.61 (second regime), positive and increasing above it, and asymptotically $\pi(r) \sim 0.8\,r$. In the operational range the reward is positive: $\pi(r) > 0$ for every $r \geq 1.0$ in both regimes, so the closed form is consistent with (2.9) wherever the protocol operates.

2.6 Summary of the Section

Table 2.5 lists the claims of this section with their status and the script that backs each.

Table 2.5: Claims of Section 2, their status, and the script that backs each.

Claim	Status	Script
Additive jumps violate positivity	Verified	`sim_positivity.py`
Multiplicative jumps preserve positivity	Verified	`sim_positivity.py`
Exact MFPT via numerical integration	Verified	`mfpt_bound_explicit.py`
$C^* = 1$ is not a valid bound	Verified	`mfpt_bound_explicit.py`
Euler bias decreases with dt	Verified	`dt_convergence.py`
Second-regime tail is approximately geometric ($P_{\text{emp}}/P_{\text{geo}} \in [0.9858, 0.9973]$); first regime has no censoring	Verified	`censoring_test.py`
Stationary distribution is Gamma(k, θ)	Verified	`stationary_stats.py`

The Section's quantitative content is limited to: positivity verification (40.58% vs 0.0000%); exact MFPT by numerical integration (11.2203 and 443.7374); implicit C^* per regime (0.562 and 3.14×10^{-8}); convergent simulated MFPT with dt (11.2453 at $dt = 5 \times 10^{-4}$); stationary moments (relative difference below 1%).

3 Tensor Compression

3.1 Tensor Representation of Reputation

Reputation is represented as a three-dimensional tensor

$$T \in [0,1]^{C \times D \times H} \tag{3.1}$$

where C indexes chains ($C = 20$ in the reference configuration), D indexes contribution domains ($D = 5$), and H indexes time horizons ($H = 5$). Entry $T[c, d, h]$ is the validator's normalized score on chain c, in domain d, at horizon h.

The tensor has $C \cdot D \cdot H = 500$ entries in the reference configuration, and the deployed data layer commits one scalar per validator: $T(i)$ denotes the entry the ingestion writes for validator i (Section 7), and the grid above is the model of what that scalar summarizes for it — chain, domain, and horizon. Admission reads $T(i)$, entry by entry: the zero-knowledge circuit of Section 4 certifies a validator's own entry against the reputational threshold Θ_r. The Frobenius norm $\|T\|_F$ over the epoch's 500 entries remains the audit quantity — it measures the whole state, including entries no validator has proved — and it is computed off the admission path.

3.2 Tucker Decomposition

The Tucker decomposition of T [19] is

$$T \approx G \times_0 A^{(0)} \times_1 A^{(1)} \times_2 A^{(2)} \tag{3.2}$$

where $G \in \mathbb{R}^{R_1 \times R_2 \times R_3}$ is the core tensor and $A^{(0)} \in \mathbb{R}^{C \times R_1}$, $A^{(1)} \in \mathbb{R}^{D \times R_2}$, $A^{(2)} \in \mathbb{R}^{H \times R_3}$ are the factor matrices. We compute the decomposition via higher-order SVD (HOSVD), which takes the leading singular vectors of each mode unfolding.

For the reference configuration we use $(R_1, R_2, R_3) = (4, 3, 3)$. The storage required for the Tucker representation is

$$R_1 R_2 R_3 + R_1 C + R_2 D + R_3 H = 146 \text{ entries} \tag{3.3}$$

versus $C \cdot D \cdot H = 500$ entries for the uncompressed tensor. The storage ratio is 3.4247× (a 70.8% reduction) [`verification/tensor/tucker_analysis.py`].

3.3 Reconstruction Error

The HOSVD is exact for tensors of rank at most (R_1, R_2, R_3) and approximate otherwise. We quantify the reconstruction error

$$\varepsilon(T) = \frac{\|T - \hat{T}\|_F}{\|T\|_F} \tag{3.4}$$

for four tensor types at rank $(4, 3, 3)$ (Table 3.1):

Table 3.1: Reconstruction error $\varepsilon(T)$ for four tensor types at rank $(4, 3, 3)$.

Tensor type	$\|T\|_F$	$\|T - \hat{T}\|_F$	$\varepsilon(T)$
Clean low-rank	73.068	0.000	4.8×10^{-16}
Low-rank + noise ($\sigma = 0.05$)	45.261	0.993	2.19%
Random (zero mean)	6.407	5.493	85.74%
Blocky (community structure)	6.775	3.559	52.53%

The four tensors are synthetic test inputs. The first two are unbounded low-rank constructions and the zero-mean random and blocky tensors live on $[-0.5, 0.5]$; none is an element of the declared domain $T \in [0,1]^{C \times D \times H}$, and their norms are not comparable to Θ_r.

Three observations follow.

First, the HOSVD implementation is exact: the clean low-rank tensor is reconstructed to machine precision (relative error 4.8×10^{-16}). This verifies the decomposition itself.

Second, the error for a low-rank tensor corrupted by noise is proportional to, and systematically below, the noise-to-signal ratio $\|N\|_F / \|T\|_F$: over 200 draws the ratio of the two lies in $[0.848, 0.938]$ with mean 0.892, because the decomposition also absorbs part of the noise. It captures the underlying low-rank component and fails to capture only the noise it does not absorb, which is by construction not low-rank.

Third: **the blocky tensor is not captured at the reference rank, but not for the reason an earlier draft gave.** A tensor constant within K communities in mode C has mode-0 rank at most K. In the reference configuration $K = 3 < R_1 = 4$, so the mode-C block structure is captured; the 52.53% error comes from truncating modes D and H to $R_2 = R_3 = 3$ while their unfoldings have rank 5. With $(4, 5, 5)$ the same noiseless tensor reconstructs to machine precision. The failure therefore depends on K relative to R_1 and on the ranks of the other modes, not on blockiness as such.

The practical implication is narrower than an earlier draft claimed: the reference rank is inadequate for this blocky instance, and the error is attributable to the other modes. Whether a different rank captures real community structure is an empirical question that depends on the correlation of validator scores across chains, domains, and horizons. We do not resolve this question in this paper.

We also do not fix a rank estimator for the noisy case. The rank $(4, 3, 3)$ is a design parameter, and the sweep of Table 3.2 is a sensitivity check, not a selection rule: no energy criterion, scree test, or parallel analysis is applied. A data-driven estimator for the noisy regime is left to future work.

3.4 Error Versus Rank

For the low-rank noisy tensor, the reconstruction error decreases monotonically with rank (Table 3.2):

Table 3.2: Storage, compression ratio and reconstruction error against rank for the low-rank noisy tensor.

(R_1, R_2, R_3)	Storage	Ratio	$\varepsilon(T)$
$(1, 1, 1)$	31	16.13×	5.02%
$(2, 2, 2)$	68	7.35×	3.24%
$(3, 3, 3)$	117	4.27×	2.32%
$(4, 3, 3)$	146	3.42×	2.19%
$(5, 5, 5)$	275	1.82×	1.52%

The trade-off is between storage reduction and reconstruction fidelity. The choice of rank is a design parameter; the reference configuration $(4, 3, 3)$ is a specific operating point, not an optimum.

3.5 Tucker Is Not in the Security Path

The Tucker compression is a storage and transmission optimization. It is not part of the trust chain of the protocol.

The zero-knowledge proof of Section 4 verifies a threshold on a validator's committed entry. It operates on the committed state, not on the Tucker approximation. A prover who fabricates the factors G and $A^{(i)}$ cannot produce a sound proof of the committed tensor's norm, because the commitment binds the prover to the raw tensor.

Consequently the storage savings of 3.42× do not translate into a reduction of the ZK circuit cost. The circuit must verify the norm of the raw tensor, which is $O(C \cdot D \cdot H)$ constraints under the commitment scheme. Tucker is used only outside the circuit, to reduce the size of the data transmitted from the reputation oracle to the verifier.

This distinction corrects a claim in an earlier formulation of the protocol, in which the storage savings were presented as a reduction of the circuit cost. The two are independent: the storage

reduction is real (3.4247×) and the circuit cost reduction is zero. The falsification log records this correction as F-012.

3.6 Summary of the Section

Table 3.3 lists the claims of this section with their status and the script that backs each.

Table 3.3: Claims of Section 3, their status, and the script that backs each.

Claim	Status	Script
HOSVD is exact for low-rank tensors	Verified	`tucker_analysis.py`
Storage reduction $500 \to 146$	Verified	`tucker_analysis.py`
Error for noisy low-rank: $\|N\|/\|T\|$	Verified	`tucker_analysis.py`
Error for random: 85.7%	Verified	`tucker_analysis.py`
Error for blocky at $(4, 3, 3)$: 52.5% (D/H truncation)	Verified	`tucker_analysis.py`
Tucker is not in the security path	Design	Sections 4, 5

The quantitative content of this Section is limited to the storage ratio and the reconstruction errors. The Tucker decomposition is a representation choice, not a security mechanism.

4 Zero-Knowledge Threshold Proof

4.1 Circuit Specification

The protocol requires each validator to prove that its own reputation entry exceeds a threshold. The entries are public (Section 7); the proof exists for succinctness, not for privacy:

$$T(i) \geq \Theta_r, \qquad i \in \{1, \ldots, 500\} \tag{4.1}$$

Equation (4.1) is the admission predicate, and it is per validator: $T(i)$ is the scalar the epoch commits to for address i, the same entry the `keccak` tree hashes for the fraud proofs of Section 7. The delivered circuit (M4d) proves that the leaf Poseidon$(i, T(i))$ is in the epoch's Poseidon root, publishes $T(i)$ in WAD as `certifiedNorm`, and constrains $0 \leq T(i) < 1$; the manager then checks `certifiedNorm` $\geq \Theta_r$. This is what connects the proof to the value $C_R(T(i))$ that Section 6 prices; a bare inequality over a proposed tensor would not.

An earlier circuit (V3) certified the global statement $\lfloor \|T\|_F \rfloor$ over the 500 entries of the epoch. It is no longer admitted by the manager: a proof of a global norm names no validator, so it is a bearer token (Section 4.2). The global norm survives as the audit quantity of Section 3, not as an admission input.

The succinctness is what the proof buys: T lives on the source chain, and the destination chain accepts the constant-size proof instead of the tensor. Section 4.5 quantifies the alternative—transmitting T to the destination chain and computing $\|T\|_F$ there—and shows that it is the more expensive path. That comparison is not against the cheapest alternative for a per-validator statement, which is a `keccak` inclusion proof of one entry for a few thousand gas; the measured cost is 306,253 gas cold (306,275 on the 2026-10-20 toolchain), and Section 10 records the gap.

The threshold Θ_r is the minimum per-validator norm required for admission. It is a design parameter, not derived from first principles in this paper. The reference configuration uses $\Theta_r = 0.5$ [CONJECTURE], and the circuit's domain $T(i) < 1$ is what makes the threshold bite over a two-to-one range.

The natural implementation is a zero-knowledge proof [8] over an R1CS circuit that verifies (4.1). The prover commits to $T(i)$ in some form and proves knowledge of a witness satisfying the inequality.

The original specification of the protocol used a Tucker-compressed witness: the prover supplies the core tensor G and factor matrices $A^{(0)}, A^{(1)}, A^{(2)}$, the circuit reconstructs $\hat{T} = G \times_0 A^{(0)} \times_1 A^{(1)} \times_2 A^{(2)}$, and verifies $\|\hat{T}\|_F \geq \Theta_r$. This specification is unsound.

4.2 Soundness Failure

The circuit accepts a proof of knowledge of some tensor $\hat{T}$ whose norm exceeds Θ_r. It does not check that $\hat{T}$ equals any committed tensor. A prover with zero reputation can therefore choose G and the factor matrices at will and produce a valid proof for any threshold.

We demonstrate the exploit by constructing a tensor with true norm below the threshold and a fabricated tensor with norm above it [`verification/zk/soundness_zk_check.py`] (Table 4.1):

Table 4.1: The exploit: the true tensor below the threshold and the fabricated tensor above it.

Tensor	$\|\cdot\|_F$	Threshold	Accepted
True reputation	0.223607	0.5	No
Fabricated (prover-chosen)	1.000000	0.5	Yes

The exploit is trivial. Groth16 [10] is sound with respect to the circuit; the circuit verifies a property of a value the prover controls completely. There is no commitment binding the prover to any specific tensor.

4.3 Repair Options

Four designs are considered, three of which are sound [`verification/zk/soundness_zk_check.py`]. We add a fifth column with the on-chain gas cost of verification, measured locally with Foundry [`integrations/avalanche/benchmarks/src/GasBenchmarks.sol`]. Table 4.2 reports the primitive costs on the EVM.

Table 4.2: Measured gas cost of the EVM primitives of the verification path.

Primitive	Gas
`ecAdd` (0x06)	292
`ecMul` (0x07)	6,142
`ecPairing` (0x08)	$45{,}000 + 34{,}000k + 142$

with k the number of pairings. Groth16 verification uses $k = 4$, giving 181,142 gas for the pairing step alone. These measurements match the EIP-1108 revision of EIP-196/197 exactly. Table 4.3 compares the designs considered.

Table 4.3: Verification cost per design: soundness, constraint count, proof size, Groth16 verification gas, and native on-chain work.

Design	Sound	Constraints	Proof size	Groth16 verify	Native on-chain work
As specified (Tucker, no commitment)	No	3,621	0.20 KB	∼206k	—
Option A: Merkle commitment, full opening	Yes	1,080,501	0.20 KB	∼206k	∼470M
Option A′: Poseidon tree over full tensor	Yes	240,261	0.20 KB	∼206k	∼30M
Option B: Pedersen + Bulletproofs	Yes	0	1.22 KB	—	∼6.5M
V3 (global norm): Poseidon sponge + norm	Yes	94,438	256 B	∼206k	—

The "Groth16 verify" column is the cost of verifying a Groth16 proof on-chain; it is approximately 206k gas with the verifying key inlined, and it is independent of circuit size. With the key in storage the same check costs 247,211 gas warm and 306,253 gas cold under the four-input key the registry installs, which is the deployed figure that Section 9.3 prices; under V3's three-input key it costs

223,403 gas warm and 286,144 gas cold, and that is the comparison Section 4 draws. The "native on-chain work" column is the cost of performing the circuit's computation natively on-chain (hashing the tensor for A and A′, scalar multiplications for B); it is not the cost of verifying a proof. The earlier draft conflated the two, which is why it read A′ as a 30M-gas alternative. The measured V3 circuit is 94,438 constraints, proves in 3.9 s, and uses a 128-byte proof (256 bytes on-chain); it is the delivered design.

The constraint count for the unsound design is the exact multiplication count of the Tucker reconstruction:

$$CR_2R_3R_1 + CDR_3R_2 + CDHR_3 + CDH + 1 = 720 + 900 + 1{,}500 + 500 + 1 = 3{,}621. \tag{4.2}$$

The first three terms are the mode products $G \times_0 A^{(0)}$, $\times_1 A^{(1)}$ and $\times_2 A^{(2)}$; the fourth is the sum of squares, and the last is the comparison. The earlier '573' counted only the core sizes and omitted the mode products.

Option A commits to T with a Merkle tree of Poseidon hashes and opens all $N = C \cdot D \cdot H$ leaves inside the circuit. The circuit requires 1,080,501 constraints. Even if a Groth16 proof of that circuit is feasible, direct verification of the proof is ∼206k gas; the ∼470M figure is the cost of performing the full opening natively on-chain, not of verifying a proof. The circuit is at the edge of practical proving and the native opening is not needed; it is not adopted.

Option A′ commits to T with a Poseidon tree over the entire tensor inside the circuit, at 240,261 constraints. The earlier claim that this is "too large for practical Groth16 proving" is withdrawn: the measured V3 Poseidon-sponge circuit is 94,438 constraints and proves in 3.9 s, and the measured V2′ circuit is 153,688 constraints and proves in 6.3 s. A 240k tree is in the same range, not an impractical outlier. Direct Groth16 verification is ∼206k gas. A′ is the natural candidate if one Poseidon Merkle root is used for both the norm proof and the per-entry fraud proofs.

Option B replaces the R1CS circuit with a Pedersen commitment to the flattened tensor and a Bulletproofs inner-product argument. The measured aggregated range proof is 1.22 KB (1,248 bytes), not the earlier 0.72 KB estimate. The R1CS constraint count is zero because there is no circuit; soundness is provided by the special-soundness of the Bulletproofs protocol and the binding property of the Pedersen commitment under the discrete logarithm assumption. Direct verification on-chain requires $2N$ scalar multiplications and $2N$ point additions, or approximately 6.5M gas for $N = 500$. This is 31× the cost of Groth16 verification, so B is not adopted.

V3 commits to T with a Poseidon sponge and certifies the floor norm in the same circuit: 94,438 constraints in the statement wrapper, a 128-byte proof (256 bytes on-chain), 3.9 s proving and ∼206k gas for direct Groth16 verification with the key inlined; 286,144 gas on the cold path under V3's own key, and 306,253 under the installed four-input key, Section 9.3. It uses no recursion, and it is no longer the admission statement the manager accepts (Section 4); the figures priced here are V3's. The data layer publishes a `keccak256` Merkle root for per-entry fraud proofs; the V3 statement carries a Poseidon root. The tie between them is the off-chain link of `src/zk/src/bridge.rs`, a recomputable commitment and not an on-chain proof. Section 10 states that trust assumption.

4.4 Direct Verification and the Recursive Option

The V3 proof is verified directly on-chain at 286,144 gas on the cold path under V3's key (223,403 warm; the ∼206k composition assumes an inlined verifying key, and the installed key's 306,253 is priced in Section 9.3). Recursion is not required for a 94k-constraint circuit, and the earlier draft's claim that it was "not optional" is withdrawn. The recursive construction is retained only as a deferred benchmark: if a future circuit needs it, Groth16 recursion over BN254 requires non-native arithmetic or a curve cycle [4, 3, 12], and neither the ∼250k gas nor the "tens of seconds" proving time is measured here. The protocol adopts V3 with direct Groth16; Tucker remains useful for data transmission outside the circuit (Section 3), but it is not in the security path.

4.5 Succinctness and the Cost of Ingesting the Tensor

The proof of Section 4 is sound by commitment binding and constant-size by construction. Because the tensor T is public, a reviewer may ask why a proof is needed at all: a validator could transmit T to the destination chain and compute $\|T\|_F = \sum_i T_i^2$ natively. We compose the cost of this alternative from two terms [`verification/zk/succinctness_comparison.py`].

First, the calldata cost of transmitting the tensor (EIP-2028: 4 gas per zero byte, 16 gas per non-zero byte). With $N = CDH = 500$ entries sent as BN254 field elements of 254 bits (one 32-byte word each), the payload is $500 \times 32 = 16{,}000$ bytes, or 256,000 gas at the upper bound. Second, the on-chain sum of squares $\sum_i T_i^2$, measured with Foundry [`integrations/avalanche/benchmarks/src/GasBenchmarks.sol`]: 428,064 gas at the reference compile, which disables the optimizer exactly as the precompile benchmarks of Section 9.3 do. The composed ingest is therefore 684,064 gas (455,430 with the optimizer enabled).

This is 2.2× the cost of verifying, directly, the Groth16 proof the registry accepts (306,253 gas on the cold path under the installed key; 286,144 under V3's, Section 9.3); direct Bulletproofs verification (∼6.5M gas, Section 4.4) is far more expensive. Transmitting the tensor and computing its norm on-chain does not dominate the constant-size proof, so the succinctness argument holds on the reference tensor. It does not, by itself, explain why the norm could not be published as a fraud-proofable leaf; that architectural question is recorded in Section 10.

Table 4.4: Gas of the Fuji deployment (chain 43113, 2026-10-18). The five creation transactions are pinned in `integrations/avalanche/scripts/fuji_20261018/`; the table is reproduced by `verification/fuji/read_fuji_gas.py`.

Contract	Gas	Cost (AVAX)
`MockBurnableERC20`	1,636,731	2.62×10^{-10}
`AVIPBurnSink`	408,820	6.54×10^{-11}
`AVIPRegistry`	2,210,748	3.54×10^{-10}
`AVIPStakeVault`	1,948,628	3.12×10^{-10}
`AVIPValidatorManager`	1,582,073	2.53×10^{-10}
Total	7,787,000	1.25×10^{-9}

4.6 Estimates and Their Provenance

The gas measurements for the EVM precompiles are exact: they match the EIP-1108 revision of EIP-196/197 to the gas unit. The Poseidon-2 measurement of 104,963 gas [`integrations/avalanche/benchmarks/src/GasBenchmarks.sol`] is an upper bound from an unoptimized Solidity implementation; production implementations are 20–40k gas. The relevant constraint on Poseidon in this paper is the R1CS count of 240 per hash, which is independent of the on-chain gas cost; the V3 sponge amortizes this to 60,748 constraints over 500 entries.

The V3 circuit, proof size and proving time are measured in `src/zk/scripts/m4b_output.txt`: 94,438 constraints, 128 bytes compressed (256 bytes on-chain), 3.9 s. The recursive SNARK cost of ∼250k gas and its "tens of seconds" proving time remain estimates and are not used by the adopted design. The Bulletproofs range proof is measured at 1.22 KB, not the earlier 0.72 KB estimate.

The exploit demonstration of Section 4 is not an estimate. The values 0.223607 and 1.000000 are computed by the script.

4.7 Summary of the Section

Table 4.5 lists the claims of this section with their status and the script that backs each.

Table 4.5: Claims of Section 4, their status, and the script that backs each.

Claim	Status	Script
Original circuit is unsound	Verified	`soundness_zk_check.py`
Fabricated tensor passes threshold	Verified	`soundness_zk_check.py`
Constraint count for unsound design: 3,621	Arithmetic	`soundness_zk_check.py`
Merkle opening: $\sim 1.08 \times 10^6$ constraints	Arithmetic	`soundness_zk_check.py`
Poseidon tree: 240,261 constraints	Arithmetic	`soundness_zk_check.py`
Bulletproofs: 0 R1CS, 1.22 KB measured	Measured	`soundness_zk_check.py`
V3 (global norm): 94,438 constraints, 3.9 s, 128 B	Measured	`src/zk/scripts/m4b_output.txt`
Groth16 $k = 4$ pairing: 181,142 gas	Measured	`GasBenchmarks.sol`
Bulletproofs direct on-chain: ∼6.5M gas	Estimated	`GasBenchmarks.sol`
Ingesting T on-chain: 684,064 gas	Measured	`succinctness_comparison.py`
Ingesting T is not cheaper than the proof	Verified	`succinctness_comparison.py`
Recursion is optional; direct V3 is viable	Design	`m4b_output.txt`

The Section demonstrates the unsoundness of the original design with a concrete exploit, and recommends a replacement. The exact cost of the replacement depends on implementation parameters not fixed in this paper; the arithmetic is correct given those parameters.

5 Collusion Detection

5.1 The Collaboration Graph

Validators interact through shared governance votes, co-authored pull requests, and simultaneous liquidity events [11, 21]. We model these interactions as a symmetric weighted graph with adjacency matrix

$$W \in \mathbb{R}^{n \times n} \tag{5.1}$$

where W_{ij} counts the interactions between validators i and j. The graph has $n = 500$ nodes in the reference configuration, with mean degree $\bar{d} = 20$.

W is an input to this analysis, not an output of the protocol. The delivered detector takes the graph as an argument (`src/rmt/src/detector.rs`) and no component of the repository derives W from T or from the ingestion of consensus events; the map from the reputation tensor to the collaboration graph is open item D-7 of the design plan. What follows therefore measures the power of the test on graphs that the protocol does not produce, and Section 10 records the gap.

The hypothesis of interest is that a subset of validators coordinates to inflate each other's reputation without genuine contribution. In graph terms, this appears as a dense subgraph (a clique) embedded in a sparser background.

5.2 The Normalization Problem

A natural spectral approach analyzes the symmetric normalized adjacency matrix

$$A_{\text{sym}} = D^{-1/2} W D^{-1/2} \tag{5.2}$$

where $D = \text{diag}(d_1, \ldots, d_n)$, $d_i = \sum_j W_{ij}$. This matrix has a bulk spectrum that recovers a Wigner-type distribution [20, 1] for heavy-tailed graphs, which the original formulation of the protocol used for detection. The spectrum of the normalized graph Laplacian for graphs of this form is analysed in [2].

Each edge is scaled by $1/\sqrt{d_i d_j}$, so the per-edge weight of an intra-clique edge is $1/27.2 \approx 0.037$ and of a background edge is $1/20.0 \approx 0.050$. A per-edge comparison alone would conclude that the clique's edges are weaker after normalization, cancelling the density signal. The relevant comparison is per-pair: intra-clique pairs are 10× more probable than background pairs, so the expected weight

is what determines the spectral signature. Quantitatively, for a clique of size $m = 20$ with intra-clique density $10\times$ the background (Table 5.1):

Table 5.1: Per-pair expected weight ratio, with and without the degree normalization.

Quantity	Value
Degree of a clique node	≈ 27.2
Degree of a background node	≈ 20.0
Normalized intra-clique edge	$1/27.2 \approx 0.037$
Normalized background edge	$1/20.0 \approx 0.050$

The clique's per-edge weight is lower after normalization, but the per-pair weight is not. Under W, the expected per-pair weight of an intra-clique pair is $10.0000\times$ that of a background pair. Under A_{sym}, with the null-model background degree cited above ($d_{\text{bg}} = 20$), the ratio is 7.4478; with the model-consistent background degree under H_1 ($d_{\text{bg}} = 27.214$), the ratio is 10.1343 [`verification/rmt/asym_weights.py`]. Under either denominator the density signal is attenuated by the normalization, not cancelled. The attenuation under A_{sym} places the clique signal inside the finite-size bulk edge, while the $10\times$ ratio under W keeps it above; the spectral table below confirms both outcomes.

We verify this numerically [`verification/rmt/clique_spectra.py`]. For a single clique of size $m = 20$ with intra-clique density $10\times$ the background, the three candidate matrices give the following leading non-trivial eigenvalue, for a single draw of the generator (seed 42) (Table 5.2):

Table 5.2: Leading non-trivial eigenvalue of the three candidate matrices in H_0 and in H_1, with the bulk edge and the gap (single draw, seed 42).

Matrix	Statistic	λ (H_0)	λ (H_1)	Bulk edge	Gap (H_1)
W (raw)	$s[-2]$	8.6896	10.6439	8.9443	+1.6996
A_{sym}	$s[-2]$	0.4254	0.4316	0.4472	−0.0156
B	$s[-1]$	8.6896	10.7011	8.9443	+1.7569

The single-seed values above are one draw from the multi-seed distribution. Across 200 seeds [`verification/rmt/clique_spectra_seeds.py`], the W gap has mean $+0.73$ with std 0.39, the A_{sym} gap has mean -0.02 with std 0.01, and the B gap has mean $+0.77$ with std 0.41. The single-seed values sit at the P99, P90 and P99 of their respective distributions. The paired claim $\text{gap}_B > \text{gap}_W$ is structural rather than informative: it follows from Weyl interlacing on $B = W - dd^\top/(2m)$. The empirical content is the mean difference $+0.0353$, small against the per-seed standard deviation.

The choice of statistic differs by matrix. For W and A_{sym}, $s[-1]$ is the Perron eigenvalue (trivial, equal to $d_{\max}$ and 1 respectively) and $s[-2]$ is the leading non-trivial direction. For B, the identity $B \cdot \mathbf{1} = 0$ places the trivial eigenvalue at the center of the spectrum, and $s[-1]$ is the signal. The change is documented in falsification log entry F-018.

The table shows that W separates the clique from the bulk (gap +1.6996 against the asymptotic edge $2\sqrt{\bar{d}} = 8.9443$). A_{sym} does not (gap -0.0156 against its bulk edge 0.4472). B separates the clique by a wider gap than W ($+1.7569$ vs $+1.6996$) and exceeds both the theoretical bulk edge and the empirically calibrated threshold $q_{99} = 9.0542$ of Section 5.4.

The relationship between B and W is explained by a structural identity. For a regular graph with degree d, the modularity matrix is $B = A - (d/n) \cdot \mathbf{1}\mathbf{1}^\top$, and since $\mathbf{1}$ is orthogonal to every other eigenvector, the spectrum of B equals that of A with the Perron eigenvalue replaced by 0. Consequently $\lambda_{\max}(B) = \lambda_2(W)$ exactly. The Erdős–Rényi graph used in the null simulation is almost regular (degrees 10–33, mean 19.88, std 4.22), so the identity holds to within 10^{-5} in H_0. In

H_1 the clique raises the degrees of its 20 members, breaking the regularity and leaving B slightly ahead of W.

This is the content of the falsification log entry F-002.

5.3 The Modularity Matrix

The matrix that resolves both problems is the modularity matrix

$$B = W - \frac{dd^\top}{2m} \tag{5.3}$$

where d is the degree vector and $m = \frac{1}{2}\sum_i d_i$ is the number of edges. The matrix B satisfies

$$B \cdot \mathbf{1} = 0 \tag{5.4}$$

which removes the degree bias by construction. Its bulk spectrum under the null is centered at zero with edge $2\sqrt{\bar{d}}$ for the degree-corrected stochastic block model.

The modularity matrix is the standard tool for community detection in the stochastic block model [5, 6]. It has the property that the leading eigenvector outside the bulk corresponds to the community structure, not to the degree distribution.

5.4 Null Distribution and Calibration

We calibrate the null distribution of $\lambda_{\max}(B)$ and $\lambda_{\min}(B)$ by Monte Carlo on Erdős–Rényi graphs with $n = 500$ and $\bar{d} = 20$ [`verification/rmt/calibration_modularity.py`]. The largest eigenvalue of a Wigner-type matrix has a Tracy–Widom limiting distribution [18]; the null quantile is calibrated by Monte Carlo rather than taken from that limit. The calibration uses $N_{H_0} = 2000$ replicates with seed 42 (Table 5.3).

Table 5.3: Null calibration of $\lambda_{\max}(B)$ and $-\lambda_{\min}(B)$ on Erdős–Rényi graphs, $N_{H_0} = 2000$ replicates.

Statistic	Mean	Std	q_{99}
$\lambda_{\max}(B)$ (right tail, cliques)	8.7679	0.1115	9.0542
$-\lambda_{\min}(B)$ (left tail, bipartite)	8.8601	0.1101	9.1371

The empirical thresholds at significance level α are (Table 5.4):

Table 5.4: Empirical decision thresholds by significance level.

α	Reject H_0 if
0.01	$\lambda_{\max}(B) > 9.0542$ or $-\lambda_{\min}(B) > 9.1371$
0.05	$\lambda_{\max}(B) > 8.9663$ or $-\lambda_{\min}(B) > 9.0428$

The dual-tail structure distinguishes clique-based collusion (right tail) from bipartite polarization (left tail). The right-tail threshold is calibrated at $q_{99} = 9.0542$, which is 2.57σ above the null mean. The left-tail threshold is calibrated at 9.1371, which is 2.52σ above its null mean.

The statistic for the right tail is $s[-1]$, the largest eigenvalue of B, not $s[-2]$. The change is documented in falsification log entry F-018: for a balanced two-block structure the signal is at $s[-1] = 17.5$ while $s[-2] = 8.6$ sits inside the bulk. The "skip Perron" convention that motivates $s[-2]$ in the adjacency matrix does not apply to the modularity matrix, where $B \cdot \mathbf{1} = 0$ places the trivial eigenvalue at the center of the spectrum.

The calibration above is performed on Erdős–Rényi graphs. The threshold does not transfer to heavy-tailed graphs. The Chung–Lu null with power-law degree distribution places every H_0 replicate above the Erdős–Rényi threshold (empirical false positive rate 1.0 at $\alpha = 0.01$ for $\beta \in$

$\{2.1, 2.3, 2.5, 3.0\}$), and per-null recalibration destroys the power against a single planted clique of size $m = 20$ with intra-clique density 10× the background (true positive rate 0.000 at $\beta \leq 2.3$, 0.155 at $\beta = 3.0$) [`verification/rmt/calibration_heavy_tail.py`]. The bulk edge $2\sqrt{\bar{d}}$ used in the calibration is not the correct edge for Chung–Lu; the entire heavy-tailed null lies above it. The direction of error is anti-conservative: the threshold flags every graph as collusive. This is recorded as falsification log entry F-019. Running the postponed experiment at $n = 5000$ does not repair the calibration (falsification log entry F-020): the Chung–Lu null places every H_0 replicate above the recalibrated Erdős–Rényi threshold ($q_{99} = 9.20$ at $n = 5000$; empirical false positive rate 1.0 for $\beta \in \{2.1, 2.3, 2.5, 3.0\}$), and $\beta = 2.1$ remains degree-capped even at that size. A clique-size sweep that holds the clique fraction fixed (falsification log entry F-021) isolates the failure from finite-size and clique-size effects: at the $n = 500$ fraction ($m = 200$, 4% of n) the Erdős–Rényi baseline recovers full power (true positive rate 1.000), whereas every Chung–Lu null stays at 0.085 or below under its own recalibrated threshold. The failure is therefore structural in the heavy-tailed null, not an artifact of n or of m: the modularity-spectrum threshold does not transfer to heavy-tailed graphs, and per-null recalibration preserves the false positive rate only by destroying the power. Detection is not the binding defence in heavy-tailed regimes; admission control there rests on the effective financial floor $\max\{S_{\min}, \Theta - C_R^{\infty}\}$ (Section 6).

5.5 Detection Power

We measure the power of the right-tail test against clique-based collusion by simulating graphs with k planted cliques of size m and intra-clique density 10× the background [`verification/rmt/calibration_modularity.py`]. The measurement uses $N_{H_1} = 500$ replicates per configuration with Wilson 95% confidence intervals. Table 5.5 reports the power per configuration.

Table 5.5: Detection power against planted cliques, by configuration.

Configuration	$\lambda_{\max}(B)$ mean	TPR @ $\alpha = 0.01$	95% CI
$k = 1$, $m = 20$	9.7452	0.972	$[0.954, 0.983]$
$k = 3$, $m = 20$	10.2714	1.000	$[0.992, 1.000]$
$k = 5$, $m = 20$	10.5294	1.000	$[0.992, 1.000]$
$k = 1$, $m = 40$	14.2441	1.000	$[0.992, 1.000]$
$k = 3$, $m = 40$	16.2281	1.000	$[0.992, 1.000]$

Detection is uniform across the five configurations. The lowest true positive rate is 0.972, for a single clique of size 20 with intra-clique density 10× the background. All other configurations are detected at 1.000.

The abrupt-transition pattern that appeared with $s[-2]$ was an artifact of the wrong statistic. With $s[-1]$, the smallest clique is detected at 97.2% and every other configuration at 100%. There is no intermediate regime in the simulated range.

The single-clique case at $m = 20$ remains the tightest: 0.972 with a Wilson 95% confidence interval of $[0.954, 0.983]$. The interval is above 0.9, so the detection is reliable. Configurations below $m = 20$ were not simulated; the boundary at which detection degrades is below the tested range.

Comparison with A_{sym}. The same test run on the normalized adjacency matrix A_{sym}, with the same clique configuration ($k = 1$, $m = 20$ on $n = 500$, $\bar{d} = 20$), gives a true positive rate of 0.033 at $\alpha = 0.01$ [`verification/rmt/archive/calibracion_rmt.py`], measured over $N_{H_1} = 30$ replicates. The modularity matrix B gives 0.972 on the same configuration, measured over $N_{H_1} = 500$ replicates. The ratio is approximately 29.2×. This is the numerical basis for the design decision to replace A_{sym} with B, documented as falsification log entry F-002.

The two Wilson 95% confidence intervals do not overlap: A_{sym} is $[0.006, 0.167]$ and B is $[0.954, 0.983]$ [`verification/rmt/wilson_intervals.py`]. The comparison is statistically significant at 95% despite the small sample size of the A_{sym} measurement ($N = 30$).

5.6 The Detection Boundary

The detection power measured in Section 5.5 is uniform across the simulated range. All five configurations have true positive rates above 0.97. There is no observable transition from partial to full detection within the tested parameter range.

This is a consequence of using $s[-1]$ as the statistic. With the previous statistic $s[-2]$, the pattern appeared to be controlled by the ratio $k \cdot m^2/n$ and a boundary around $k \cdot m^2/n \approx 1.5$ was conjectured. That pattern was an artifact: with $s[-1]$ the same configurations all sit above the (much lower) effective boundary.

The detection boundary for the modularity-spectrum test with $s[-1]$ on the degree-corrected stochastic block model lies below the configurations tested here [CONJECTURE]. We do not locate it precisely; the smallest configuration tested (a single clique of size $m = 20$) is detected at 0.972, and configurations smaller than $m = 20$ were not simulated. A derivation of the boundary from the Kesten–Stigum threshold or from the random matrix literature would be a separate contribution.

Consequently, collusion at the scale tested here is detectable from the collaboration graph alone. Collusion below the (unlocated) boundary would not be. The protocol relies on economic deterrence through slashing (Section 7) and the decay of reputation (Section 2) for the sub-boundary regime, which is not characterized in this paper.

5.7 Limitations

The calibration is performed on Erdős–Rényi and Chung–Lu graphs. Real collaboration graphs have additional structure not captured by these models: community hierarchies, temporal correlations between interactions, and directionality (who initiates contact). The empirical quantiles in Section 5.4 are valid for the calibration distribution; their validity on real graphs is an open question.

The heavy-tailed case is not merely an open question but a negative result. The modularity-spectrum test does not transfer to heavy-tailed collaboration graphs: the failure persists at $n = 5000$, and a clique-size sweep shows it is structural rather than an artifact of n or of the clique size (falsification log entries F-019, F-020, F-021). In those regimes detection is not the binding defence. What the protocol then relies on is the minimum admission cost of Section 6 — $S_f^*(T) \geq \Theta - C_R(1) = 0.6$ over the domain the circuit certifies, with $S_{\min} = 0.2\Theta$ as the floor of the unbounded model, one fifth of the pure-PoS stake per node — together with slashing (Section 7) and the decay of reputation (Section 2), not on the spectral test. That floor is a price of entry, not a guarantee of security: it bounds what a node must post, not what a cartel can gain, and Section 6 does not price the cost of acquiring the reputation that earns the discount.

The left tail ($-\lambda_{\min}$) is measured on disassortative block structures [`verification/rmt/bipartite_power.py`]. The true positive rate is 0.990–1.000 across block configurations from balanced (250/250) to heavily unbalanced (50/450), with density ratios $\rho_{\text{between}} \in \{2, 5, 10\}$ and $N_{H_1} = 500$ replicates. The left-tail test has power comparable to the right-tail test.

An adversarial validator who knows the test can attempt to avoid detection by keeping the clique size below the (unlocated) boundary, mixing clique members with honest interactions, or avoiding clique structures entirely. The test has power against the naive adversary (uniform cliques and bipartite structures of moderate size); its power against an adaptive adversary below the boundary is not characterized.

5.8 Summary of the Section

Table 5.6 lists the claims of this section with their status and the script that backs each.

Table 5.6: Claims of Section 5, their status, and the script that backs each.

Claim	Status	Script
A_{sym} masks cliques	Verified	`clique_spectra.py`, `sim_laplacian.py`
Modularity B with $s[-1]$ detects cliques	Verified	`calibration_modularity.py`
Null q_{99} (right tail, $s[-1]$) = 9.0542	Verified	`calibration_modularity.py`
TPR $k = 1$, $m = 20 = 0.972$	Verified	`calibration_modularity.py`
TPR $k = 3$, $m = 20 = 1.000$	Verified	`calibration_modularity.py`
TPR $k = 1$, $m = 40 = 1.000$	Verified	`calibration_modularity.py`
A_{sym} vs B ratio = 29.2×	Measured (point estimate; CI reported)	`wilson_intervals.py`
Detection boundary below tested range	Conjecture	—
Left-tail power measured	Verified	`bipartite_power.py`

The Section's quantitative content is limited to the calibration of the null distribution, the power measurements for both tails, and the demonstration that A_{sym} fails where B succeeds.

6 Payment Rule

6.1 The Revelation Problem

A validator is admitted to the validator set if its hybrid collateral exceeds the threshold:

$$\text{CoA}(i) = S_f(i) + C_R(T(i)) \geq \Theta(t) \tag{6.1}$$

where S_f is the financial stake, C_R is the reputation-weighted access cost, and $T = T(i)$ is the validator's committed entry.

The allocation of collateral between its financial and reputational components depends only on T. It does not depend on any private information that the validator might hold. This simplifies the mechanism design problem relative to a classic auction, but it also means that the standard tools of auction theory do not apply directly.

We address the two candidate sources of private information below.

6.2 Verified Reputation

The reputation tensor T is designed to be verifiable on-chain via the zero-knowledge threshold proof of Section 4; the V3 circuit is measured, but the on-chain verifier is not deployed. Under that specification, a validator whose reputation exceeds the threshold can prove it, and a validator whose reputation falls below the threshold cannot produce a valid proof.

Consequently, T is not a private type in the sense of mechanism design. The validator cannot misreport the leaf: the statement proved under the V3 repair of Section 4.4 opens the Poseidon leaf $\text{Poseidon}(i, r)$ in the epoch's committed root with $0 \leq r < 1$, and the registry builds the statement from its own storage with the validator address as a public input, so a proof that admits one address cannot be replayed for another. That closes the bearer-token defect of the earlier statement, which named no validator. Two limits remain, and both are stated in Section 10: the certificate binds an entry, not a validator's tenure, and the on-chain verifier is not deployed, so the path is specified and measured but not exercised on-chain. The revelation principle does not apply because there is nothing to reveal. The claim is local to T: it does not assert that validators have no private information. Entry timing, key rotation, and channel-aggression strategies can be private and can affect revenue, and the mechanism neither elicits nor prices those dimensions.

6.3 Private Cost of Capital

The validator's cost of capital $\omega \in [\underline{\omega}, \bar{\omega}]$ is private. It represents the opportunity cost of locking financial stake. A validator with low ω can lock stake cheaply; a validator with high ω faces a higher opportunity cost.

The allocation rule is

$$S_f^*(T) = \max\{S_{\min}, \Theta(t) - C_R(T)\} \tag{6.2}$$

which does not depend on ω. Here T is the validator's own entry, $r = T(i)$, so $C_R(T) \equiv C_R(r)$: the allocation is a function of that scalar. The type affects the validator's utility but not its allocation.

The floor $S_{\min}$ is the technical minimum financial stake, independent of reputation. It is a design parameter, not a value at risk in the financial sense, and it is not arbitrary: it equals the complement of the collateral saturation, $S_{\min} = \Theta - C_R^\infty$ with $C_R^\infty = \lim_{r\to\infty} C_R(r)$. For the reference value function $C_R(r) = 0.8\,r/(1+r)$ the collateral saturates at $C_R^\infty = 0.8$, so $S_{\min} = 1.0 - 0.8 = 0.2$. The floor is the largest minimum stake that stays inactive over the unbounded reputation domain: the unfloored allocation $\Theta - C_R(T)$ has infimum $\Theta - C_R^\infty = 0.2$, which the floor meets but never exceeds [`verification/mechanism/financial_floor_sensitivity.py`]. A sweep of the saturation C_R^∞ over $[0.5, 1.0]$ moves $S_{\min} = \Theta - C_R^\infty$ over $[0.0, 0.5]$; the reference value 0.2 corresponds to $C_R^\infty = 0.8$, i.e. reputation supplies at most 80% of the admission threshold. Those are the values of the unbounded domain. The delivered circuit bounds the certified norm at $r < 1$ (Section 4), so over the domain admission can actually read, $C_R(r) < C_R(1) = 0.4$ and the unfloored stake is at least $\Theta - C_R(1) = 0.6$. The floor $S_{\min} = 0.2$ therefore never binds: the floor the deployed rule applies to an admitted validator is 0.6, and it is set by the circuit's domain rather than by the saturation argument.

The argument of C_R in (6.2) is the validator's own entry, $r = T(i)$, with C_R the value function of Section 2. It is not the Frobenius norm of the epoch's tensor: that quantity aggregates entries that need not be admitted, and it is computed off the admission path (Section 10). The floor $S_{\min} = 0.2$ is the financial component of admission; the model does not price the cost of acquiring the reputation that earns the credit, so (6.2) is not a network-level cost-of-attack bound.

The utility of a validator of type ω with reputation T is

$$U(\omega, T) = \pi_{\text{val}} - \omega \cdot S_f^*(T) - F_0 \tag{6.3}$$

where π_{val} is the expected validation revenue per epoch and F_0 is the fee charged for participation.

6.4 The Uniform Posted Price

Since the allocation does not depend on ω, the only mechanism design decision is the participation fee F_0. We choose a uniform posted price: all validators pay the same F_0, regardless of type.

The optimal F_0 is the largest value that preserves Individual Rationality for all types. The worst-case type is $(\bar{\omega}, T = 0)$, which combines the highest cost of capital with the lowest reputation:

$$U(\bar{\omega}, 0) = \pi_{\text{val}} - \bar{\omega} \cdot \Theta - F_0 \geq 0 \tag{6.4}$$

Solving for F_0 at equality:

$$F_0^* = \pi_{\text{val}} - \bar{\omega} \cdot \Theta \tag{6.5}$$

With the reference parameters ($\pi_{\text{val}} = 0.15$, $\bar{\omega} = 0.10$, $\Theta = 1.0$):

$$F_0^* = 0.15 - 0.10 \cdot 1.0 = 0.0500 \tag{6.6}$$

The fee is charged per epoch and is the same for all participating validators.

6.5 Individual Rationality

The posted price F_0^* saturates IR at the worst-case type by construction. We verify numerically that IR holds for all types in the support and that the saturation is exact [`verification/mechanism/fee_specification.py`].

The Monte Carlo draws $N_{\text{VALIDATORS}} = 1000$ validators with $\omega \sim U[0.01, 0.10]$ and r uniform on $[0, 3]$. The draw is the domain of the unconstrained model, not of the circuit, which certifies

$r \in [\Theta_r, 1)$: the table's $U_{\max}$ and U_{mean} price an $r > 1$ region that cannot be proved, and its $U_{\min}$ prices an $r < \Theta_r$ region the manager does not admit. The table is not recomputed for the restricted domain. Individual rationality holds at the worst point of that domain, $r = \Theta_r$ with $\omega = 0.10$: $S_f^* = 0.7333$ and $U = 0.0267$. The measured utilities are (Table 6.1):

Table 6.1: Measured utilities of the sampled validator population under the posted price.

Statistic	Value
$U_{\min}$	0.009747
U_{mean}	0.069152
$U_{\max}$	0.095891
Fraction with $U \geq 0$	1.0000

The analytical saturation $U(\bar{\omega}, 0) = 0$ is exact, with residual below 10^{-9}. The Monte Carlo $U_{\min}$ is strictly positive (0.009747) because the specific pair $(\omega = 0.10, T = 0)$ is not sampled exactly in a finite draw. The bound holds analytically for the worst-case type and empirically for the sampled population.

6.6 Sensitivity

If the fee exceeds F_0^*, IR is violated for the worst-case types. We verify this numerically (Table 6.2):

Table 6.2: Individual rationality and revenue against the fee, relative to the optimal F_0^*.

F_0	Fraction with $U < 0$	Gross revenue	Net revenue
0.025	0.0000	25.00	25.00
0.040	0.0000	40.00	40.00
0.050	0.0000	50.00	50.00
0.060	0.0020	60.00	59.88

At $F_0 = 0.06$ (20% above F_0^*), 0.2% of the sampled validators violate IR. The gross revenue trade-off is linear: increasing the fee above F_0^* gains revenue at the cost of excluding the worst-case types. Net of the excluded validators, the revenue at $F_0 = 0.06$ is 59.88 rather than 60.00.

6.7 Incentive Compatibility

Incentive Compatibility is tautological in this mechanism. There is no private type to misreport: T is verified on-chain, and $S_f^*(T)$ does not depend on ω. A validator cannot improve its allocation by misreporting, because the allocation is determined by a public, verifiable signal. The tautology is limited to the reporting of T; it is not a claim of full incentive compatibility over entry timing, key rotation, or channel behavior (Section 6.2).

The falsification log records this as F-006. The original formulation of the protocol applied Myerson's optimal auction rule [14], which requires a private one-dimensional type. The current mechanism does not have one. The payment rule is a saturated uniform posted price, not a Myerson auction.

6.8 Summary of the Section

Table 6.3 lists the claims of this section with their status and the script that backs each.

Table 6.3: Claims of Section 6, their status, and the script that backs each.

Claim	Status	Script
T is verified, not private	Design	Section 4
$S_f^*(T)$ independent of ω	Design	—
$S_{\min} = \Theta - C_R^{\infty} = 0.2$	Analytical	`financial_floor_sensitivity.py`
Effective floor over the circuit's domain ($r < 1$): $\Theta - C_R(1) = 0.6$	Analytical	—
$F_0^* = \pi_{\text{val}} - \bar{\omega}\Theta$	Analytical	`fee_specification.py`
IR holds for all types	Verified	`fee_specification.py`
IR saturation exact at worst-case type	Verified	`fee_specification.py`
$F_0 = 1.2F_0^*$ violates IR for 0.2%	Verified	`fee_specification.py`
IC is tautological in T; not in entry timing	Logical	—

The payment rule is a uniform posted price, not a Myerson auction. The absence of a private type with respect to the allocation rule makes the mechanism simpler than standard auction theory would suggest.

7 Data Layer

7.1 Ingestion of Consensus Events

The reputation tensor T introduced in Section 3 is not declared by the validator. It is derived from consensus events on the source chains [17]. For each chain c, domain d, and horizon h, the entry $T[c, d, h]$ is computed from a formula frozen at genesis:

$$T[c, d, h] = f_c(\text{events in window } h) \tag{7.1}$$

where f_c is a public function that maps the observed consensus events to a normalized score in $[0, 1]$. The formula is not changeable by the validator; it is fixed before deployment and applies identically to all validators on chain c.

The paper does not fix a numerical f_c. The reference implementation declares a `GenesisFormula` instance (chain 43113, weights $(att, gov, liq) = 3/5/2$, scale 1000, genesis digest `0x2fcd03b8...`); that instance is not the paper's f_c, and the tensor experiments of Section 3 are synthetic. All quantitative claims are therefore conditional on the genesis-frozen f_c.

The events themselves are consensus outputs: signed attestations, governance votes, and liquidity events that have been finalized on the source chain. They are not subjective judgments or off-chain declarations.

7.2 Transport and Freshness

The commitment to the reputation tensor T is produced by the data layer, not by the validator. The data layer reads the source-chain events, applies the public ingestion formula f_c of Section 7, and publishes the commitment optimistically. Soundness rests on the fraud proof and slashing mechanisms of the data layer, not on the validator's honesty.

For Avalanche L1 source chains, the commitment root is propagated via Avalanche Warp Messaging through the Teleporter contract, with the source chain's quorum as the trust anchor. For non-Avalanche source chains, the transport requires a light client or a zero-knowledge bridge.

The tensor T is frozen per epoch. Two consecutive epochs cannot differ in more than $\gamma \cdot N$ entries, where $N = CDH$ and γ is a freshness parameter. The constraint bounds the number of changed entries, not their magnitude; Section 7.6 states the norm caveat.

7.3 The Fraud Proof Mechanism

Any party can challenge a published entry by re-deriving it: the reporter publishes the consensus events behind $T[c, d, h]$, and the adjudicator re-executes the same public, genesis-frozen ingestion formula f_c and compares its result with the published leaf. The challenge succeeds when the

two disagree, and only then: the entry is adjudicated fraudulent and the honest challenger is owed the bounty B of the conditional model below. A pair of contradictory openings is not a usable proof against the tensor's own commitment, because one leaf under one Merkle root has one value — no set of inclusion proofs can exhibit a contradiction — so the adjudicator checks the one thing that can fail, the value. Both the bounty B and the slashing penalty S_{slash} are model parameters of this section: the delivered data layer adjudicates and reports, and does not move funds. Two subjects that sentence leaves implicit are delivered with it, and they are what make a loss executable: the validator manager, which prices the admission and debits the escrow [`integrations/avalanche/contracts/AVIPValidatorManager.sol`], and the custody vault that holds that escrow [`integrations/avalanche/contracts/AVIPStakeVault.sol`]. The data layer reports; the manager is what acts on the report.

We model the audit equilibrium as follows. Let N_{ATT} be the number of attestations per epoch, p_{fraud} the fraction that are fraudulent, and c_{audit} the cost of auditing one attestation. The bounty is a single common pool and not a reward per auditor: it is divided among the auditors who are active, so the income of an individual auditor falls as participation grows. Each verifier decides whether to audit based on the expected revenue [16]:

$$\mathbb{E}[\text{revenue}] = \frac{p_{\text{fraud}} \cdot N_{\text{ATT}} \cdot B}{n_{\text{auditors}}} - N_{\text{ATT}} \cdot c_{\text{audit}} \tag{7.2}$$

where n_{auditors} is the number of active auditors. A verifier enters if $\mathbb{E}[\text{revenue}] > 0$ and exits otherwise. Under the exogenous p_{fraud} assumption, the entry condition yields the interior solution

$$n^*(B) = \left\lfloor \frac{B}{B^*} \right\rfloor \tag{7.3}$$

with the threshold fee

$$B^* = \frac{c_{\text{audit}}}{p_{\text{fraud}}} \tag{7.4}$$

For $B < B^*$ no auditor enters; for $B \geq B^*$ at least one auditor is active. The threshold B^* is the fee at which a single auditor is indifferent between auditing and not. Detection in this model is a step and not a probability: an auditor reviews every attestation of the epoch, so coverage is 1 whenever $n_{\text{auditors}} \geq 1$ and 0 otherwise.

7.4 Numerical Verification

We evaluate (7.3) and (7.4) by Monte Carlo [`verification/data-layer/simulate_data_layer_v3.py`], with one caveat about what the exercise can show: the script computes $n^*(B)$ from (7.3) and feeds it into the epoch loop as a premise, so the simulation measures the audits and the integrity that the assumed n^* implies, and does not test whether n^* auditors would in fact enter. The reference parameters are $N_{\text{ATT}} = 500$, $c_{\text{audit}} = 0.10$, $p_{\text{fraud}} = 0.10$, $V_{\text{fraud}} = 1.0$, $S_{\text{slash}} = 10.0$, $N_{\text{VERIFIERS}} = 20$. The threshold is $B^* = 0.10/0.10 = 1.0$. Table 7.1 reports the conditional points at $p_{\text{fraud}} = 0.10$. It is not an equilibrium in p_{fraud}: if the attacker chooses p_{fraud}, the negative profit at $B \geq B^*$ drives $p_{\text{fraud}} \to 0$, which drives $B^* \to \infty$; that loop is recorded in Section 10.

Table 7.1: Audit comparative statics at exogenous $p_{\text{fraud}} = 0.10$: auditors, detection, attacker and verifier profit, integrity. Verifier profit is per active auditor under equal division and full participation; free-riding and the endogeneity of p_{fraud} are not represented (Section 10).

B	$n^*(B)$	Detection	Attacker profit	Verifier profit (per auditor)	Integrity
0.75	0	0.000	+49.91	0.00	0.9002
1.00	1	1.000	−502.00	+0.20	1.0000
2.00	2	1.000	−502.00	+0.20	1.0000
5.00	5	1.000	−502.00	+0.20	1.0000
10.00	10	1.000	−502.00	+0.20	1.0000
20.00	20	1.000	−502.00	+0.20	1.0000

The table reports the interior points of the conditional model, where B is either below the threshold or an exact integer multiple of B^*. The transition at $B = B^*$ is exact. The verifier-profit column divides the pool among the n^* active auditors and assumes that every one of them audits; free-riding would change the individual payoff and is not modeled, so the column is conditional on full participation, not an equilibrium prediction (Section 10). For B below the threshold no verifier audits; the attacker earns $V_{\text{fraud}} \cdot N_{\text{ATT}} \cdot p_{\text{fraud}}$ per epoch without friction. For B at or above the threshold the auditor coverage reaches 100% in this model, and the attacker's expected profit is negative because the slashing dominates the fraud gain.

At the interior equilibrium points the analytical verifier profit is zero: the per-auditor revenue $D \cdot B/n^*$ equals the per-auditor cost $N_{\text{ATT}} \cdot c_{\text{audit}} = 50$ exactly when $n^* = B/B^*$. The observed +0.20 is the Monte Carlo deviation from zero, equal to $(D_{\text{obs}} - D_{\text{theory}}) \cdot B/n^*$. With $D_{\text{obs}} \approx 50.2$ and $D_{\text{theory}} = 50.0$, the deviation is $0.2 \cdot B^* = 0.20$. Between interior points the verifier profit is strictly positive; the value depends on B and is tabulated in the script output.

7.5 Slashing and Integrity

The attacker profit is negative for all $B \geq B^*$ because the slashing parameter $S_{\text{slash}} = 10.0$ exceeds the fraud gain $V_{\text{fraud}} = 1.0$ by an order of magnitude. Both figures are constants of the worksheet [`verification/data-layer/simulate_data_layer_v3.py`], and neither is a protocol quantity: the delivered manager debits the validator's whole recorded stake when it slashes, and the amount is no longer a parameter of the call at all [`integrations/avalanche/contracts/AVIPValidatorManager.sol`]. The loss that is actually collateralized on-chain is therefore the stake that was posted, which is at least $S_f^*(T)$ — over the range the circuit admits, at least $\Theta - C_R(1) = 0.6$ — and unbounded above, because nothing in the admission path caps the stake a validator may post. The bound $L \leq \Theta = 1.0$ of this model is a coverage convention, not an enforced cap. $S_{\text{slash}} = 10.0$ appears in no transferred amount; the attacker and verifier profits of Table 7.1 are that worksheet's bookkeeping. The design of the amount is outside the scope of this section.

The slash debits the escrow and revokes the admission; it writes no reputation state, so the reduction of Section 2 is a property of the reputation process and not a consequence of the slash. What the mechanism adds beyond the debit is an executable re-entry cost: an ejected validator that re-declares the same norm pays a surcharge of $\kappa = 0.1$ per ejection on top of $S_f^*(T)$, charged in the admission call itself [`integrations/avalanche/contracts/AVIPValidatorManager.sol`]. The surcharge is priced on the validator's own ejection count and is never folded into S_f^*, so the collateral image $\Theta - C_R(T)$ stays a function of the certified norm and cannot go negative. Escrowed loss and this re-entry cost together are what make fraud unprofitable in this model for any B above the threshold; whether the executable loss on its own deters an attacker of that size is the question Section 10 leaves open.

Once the threshold is crossed, the model's deterrence rests on a condition that the step convention satisfies without stating it. A risk-neutral attacker runs the fraud only while its expected profit is positive, $E = (1 - q) \cdot G - q \cdot L > 0$, which is to say while the per-epoch detection probability stays

below $G/(G+L)$. Deterrence is the complement of that condition:

$$q > \frac{G}{G+L} \tag{7.5}$$

with G the gain if undetected and L the loss if detected. We replace the worksheet's scalar gain with the quantities the protocol carries, which also names the attacker. The attacker is the publisher of the fraudulent leaf. In v1 that publisher is the data layer's sequencer, not the validator: the manager can debit a validator's escrow only when the leaf is attributed to that validator, and v1 does not implement that attribution. The model therefore prices the publisher's incentive; the executable validator slash remains future work. The publisher inflates the certified norm r_{fraud} above its true one: the admission rule then charges it $S_f^*(r_{\text{fraud}})$ instead of $S_f^*(0) = \Theta$, and the gain is the stake the declared norm buys, $\Delta S = S_f^*(0) - S_f^*(r_{\text{fraud}}) = C_R(r_{\text{fraud}})$, per undetected epoch, at the private cost of capital ω of Section 6: $G(k, \omega) = k \cdot \omega \cdot \Delta S$ over k undetected epochs. This is the opportunity cost of capital avoided; if the attacker's outside option is not validating, the relevant gain is validation income, which the model does not estimate. The loss is the escrow actually debited, at least $S_f^*(r_{\text{fraud}})$ because the manager takes the whole stake on record, plus the re-entry surcharge of the ejection count. The detection latency k therefore enters the condition as an explicit constraint rather than as a constant: [`verification/mechanism/m05_attack_model.py`] evaluates (7.5) over k, ω and the certified norm. At the reference inflation $r_{\text{fraud}} = 1$ and the worst-case type $\omega = 0.10$, the required probability is $q > 0.0625$ at one epoch of latency, $q > 0.25$ at five, and $q > 0.375$ at nine — the largest k that the freshness bound of Section 7.6 admits. These are the $r_{\text{fraud}} = 1$ values, and under the delivered circuit they are also the worst case: the certified norm is bounded below 1, so $C_R < 0.4$, $S_f^* > 0.6$ and $G(k) < 0.04k$ — an inflation beyond 1 is not provable. The unbounded-limit values $q > 0.2857$ at $k = 1$ and $q > 0.7826$ at $k = 9$ belong to the unconstrained model and lie outside the certified domain. The step convention still holds the first of those with room to spare: coverage of 1 means detection within the audited epoch, and the margin is $L/G = 15$ against the threshold it must clear. What the convention does not supply is the latency itself, and that is what the freshness parameter bounds.

7.6 Freshness as Second Line of Defense

The freshness parameter γ bounds the propagation of undetected fraud across epochs. If a fraudulent entry is present in epoch t, it can propagate to epoch $t+1$ only within the γ fraction allowed. Over k epochs, the maximum fraction of fraudulent entries in the tensor is bounded by

$$\text{fraud fraction} \leq \gamma \cdot k \tag{7.6}$$

in the worst case, where the adversary can insert fresh fraud each epoch. The growth is linear, not exponential, and the bound constrains only while $\gamma \cdot k < 1$: the example pair $\gamma = 0.10$ with $k = 10$ epochs gives $\gamma \cdot k = 1.0$, where the maximum fraudulent fraction is 1.0 — the whole tensor — so there the bound permits everything and constrains nothing. Below that threshold the constraint bites: $\gamma = 0.05$ gives a maximum of 0.5 after 10 epochs, and $\gamma = 0.02$ gives 0.2. While $\gamma \cdot k < 1$ the freshness constraint prevents a single undetected fraud from contaminating the entire tensor. This is also the constraint that bounds the detection latency of (7.5): the same k is the number of epochs an undetected fraud can run, so γ fixes the largest window the detection probability must survive. For the reference pair $\gamma = 0.10$ that window is nine epochs, and the probability the worst-case type requires there is 0.375, against the 0.0625 of a single epoch [`verification/mechanism/m05_attack_model.py`]. The freshness parameter is therefore the second line of defense in a precise sense: where the audit fails, freshness is what limits the loss, and it does so by capping k rather than by reducing q. The bound is on the number of changed entries, not on their magnitude: 50 entries moving from 0 to 1 raise $\|T\|_F$ to $\sqrt{50} = 7.07$ and C_R to approximately 0.70 in one epoch. A norm-based freshness rule is future work. F-022 records the detection-latency model and the block-time calibration that parametrise the deterrence condition over k.

7.7 Summary of the Section

Table 7.2 lists the claims of this section with their status and the script that backs each.

Table 7.2: Claims of Section 7, their status, and the script that backs each.

Claim	Status	Script
$B^* = c_{\text{audit}}/p_{\text{fraud}}$	Analytical	`simulate_data_layer_v3.py`
$n^*(B) = \lfloor B/B^* \rfloor$ equilibrium	Analytical	`simulate_data_layer_v3.py`
Attacker profit -502 with $S_{\text{slash}} = 10$	Analytical	`simulate_data_layer_v3.py`
Integrity $= 1.0$ when $n^* \geq 1$	Design	`simulate_data_layer_v3.py`
Freshness bounds propagation linearly	Analytical	—

The Section describes the ingestion, transport, and fraud proof mechanism of the data layer, and verifies the audit equilibrium by Monte Carlo. The freshness parameter provides a second line of defense in case the fraud proof mechanism fails to detect a particular attack.

8 Bootstrap Dynamics

8.1 The Genesis Problem

At the genesis block, no validator has reputation. The reputation tensor T is identically zero and the reputation-weighted access cost $C_R(0)$ is zero. The protocol therefore begins as a pure proof-of-stake system with full financial stake requirements, and transitions to a hybrid regime as validators accumulate reputation.

The bootstrap question is: does the transition concentrate reputation in the early validators, and if so, how much? A mechanism that concentrates reputation in a small set of genesis validators would recreate the centralization it was designed to avoid.

8.2 Reputation Models

The dynamics of reputation during the bootstrap determine the outcome. We compare three models in earlier work and select one for the paper. The models differ in how a validator's activity a translates into reputation growth (Table 8.1):

Table 8.1: The three bootstrap models: gain rule and stationary reputation.

Model	Gain rule	Stationary value
Saturating	$k \cdot a \cdot (1 - R)$	$R \to 1$ for all a
Decreasing	$k \cdot a \cdot (1 - R)^2$	$R \to 1$ for all a
net_gain	$g \cdot a \cdot (1 - R) - \delta R$	$R^* = ga/(ga + \delta)$

The saturating and decreasing models converge to $R \to 1$ for all validators regardless of activity. After a sufficiently long horizon, the reputation is identical for all validators and carries no information about contribution. This is the content of the earlier comparison [`verification/bootstrap/compare_reputation_models.py`].

The net_gain model has a stationary value that depends on activity:

$$R^* = \frac{g \cdot a}{g \cdot a + \delta} \tag{8.1}$$

The bootstrap operates on a bounded reputation score $R \in [0, 1)$, distinct from the unbounded state r of Section 2; the two are related by $R = r/(1+r)$. During the bootstrap phase (from $R(0) = 0$, before the protocol reaches its stationary regime) the value function $C_R(r)$ of Section 2 is replaced

by the bootstrap-specific form $\tilde{C}_R(R) = 0.8\,R/(1+R)$. This is a model choice, not the Section 2 function under the map: substituting $r = R/(1-R)$ into $C_R(r) = 0.8r/(1+r)$ gives $0.8R$, not $\tilde{C}_R(R)$. All bootstrap numbers in this section are computed under $\tilde{C}_R$; the value function $C_R(r)$ applies to the post-bootstrap steady state, and the two should not be equated.

The correlation between activity and reputation at stationarity is 0.76 in the reference simulation, versus 0.08–0.11 for the saturating models [`verification/bootstrap/compare_reputation_models.py`]. This correlation is the declared model-selection criterion: net_gain is selected because its stationary reputation tracks contribution, and the saturating models fail that property by an order of magnitude. The sweep over δ at fixed $g = 0.01$ [`verification/bootstrap/sweep_net_gain.py`] gives a maximum of R_{std} at $\delta/g = 1.5$. We use $g = 0.01$, $\delta = 0.015$. The sweep fixes the operating point; it is not a selection test and no threshold on R_{std} is used to choose the model.

The quantity R_{std} is the cross-sectional sample standard deviation of the reputation score R over the active validators at the final epoch of the simulation, with the population convention (numpy default `ddof = 0`); the reference simulation runs $N = 100$ validators over $T = 500$ epochs [`verification/bootstrap/r_std_specification.py`].

8.3 Bootstrap Simulation

We simulate the bootstrap from $R(0) = 0$ over 500 epochs [`verification/bootstrap/simulate_bootstrap_net_gain.py`]. Parameters:

- $N_{\text{MAX}} = 100$ validators, $N_{\text{INIT}} = 10$ at genesis.
- Entry rate 0.03 per epoch for new validators.
- Activity $a \sim \text{LogNormal}(0, 0.5)$ per validator.
- Transition rate $\tau(t) = \min(1, t/200)$, reaching 1 at epoch 200.

We compare three policies:

- **pure_pos**: $S_f = \Theta$ throughout. No reputation adjustment.
- **hybrid_no_decay**: $S_f(t) = \max\{S_{\min}, \Theta - \tau(t) \cdot C_R(R(t))\}$.
- **hybrid_decay**: same as hybrid_no_decay with R decaying at half-life 50 epochs.

8.4 Results

Table 8.2 reports the resulting allocation policies.

Table 8.2: Allocation policy under each bootstrap model: Gini, top-ten concentration and reputation statistics.

Policy	$t(\tau{=}1)$	Gini	Top-10	S_f mean	R mean	R max
pure_pos	500	0.1283	0.1367	1.0000	0.3936	0.5976
hybrid_no_decay	200	0.1283	0.1367	0.7764	0.3936	0.5976
hybrid_decay	200	0.1563	0.1477	0.8393	0.2553	0.4348

The S_f mean column measures the mean financial stake required at the end of the simulation. Under pure_pos it is $\Theta = 1.0$. Under hybrid_no_decay it is 0.7764, a reduction of 22.4%. Under hybrid_decay it is 0.8393, a reduction of 16.1%. These are means, not medians; the corresponding medians are 0.7714 and 0.4000 for the recommended policy.

The R mean column is the mean reputation. Under net_gain it converges to the stationary value $R^* = ga/(ga + \delta)$. For unit activity $a \approx 1$, $R^* \approx 0.4$, consistent with the measured mean 0.3936.

The Gini and Top-10 columns are identical between pure_pos and hybrid_no_decay because both policies track the same reputation trajectory $R(t)$. The difference between them is in the financial stake policy $S_f(t)$, not in the reputation. Gini and Top-10 are computed on the distribution of R, not on S_f.

8.5 Comparison of Policies

Two observations follow from the table.

Decay does not reduce concentration. The hybrid_decay policy has higher Gini (0.1563) than hybrid_no_decay (0.1283). This is the opposite of the intuition that decay limits accumulation. The reason is that decay lowers the ceiling ($R_{\max}$ drops from 0.5976 to 0.4348) but also lowers the floor, spreading the distribution. Under hybrid_no_decay, the top validators saturate near $R \approx 0.6$ and the distribution compresses; under hybrid_decay, they fall back to $R \approx 0.43$ and the distribution widens. The falsification log records this as F-015.

hybrid_decay is dominated on both metrics. It has higher Gini (0.1563 vs 0.1283) and higher mean S_f (0.8393 vs 0.7764). The only advantage is lower $R_{\max}$, which could matter if the protocol has a constraint on maximum reputation. Absent such a constraint, hybrid_no_decay is the recommended policy.

8.6 Recommendation

The protocol uses hybrid_no_decay. The financial stake reduction under this policy is 22.4% at the mean validator. This is lower than the 37% that an earlier analysis reported under the saturating model [see falsification log entry F-016], because the saturating model produces $R \to 1$ for all validators while net_gain produces $R^* \approx 0.4$.

The reduction is real but modest. It corresponds to a validator with mean reputation needing to lock 77.6% of the stake it would need under pure proof-of-stake. Validators with above-mean reputation lock less; validators below mean lock more.

8.7 Summary of the Section

Table 8.3 lists the claims of this section with their status and the script that backs each.

Table 8.3: Claims of Section 8, their status, and the script that backs each.

Claim	Status	Script
net_gain is the selected model	Verified	`compare_reputation_models.py`
$\delta/g = 1.5$ maximizes R_{std}	Verified	`sweep_net_gain.py`
S_f mean 0.7764 under hybrid_no_decay	Verified	`simulate_bootstrap_net_gain.py`
22.4% stake reduction at mean	Verified	`simulate_bootstrap_net_gain.py`
hybrid_decay raises Gini	Verified	`simulate_bootstrap_net_gain.py`
Hybrid_no_decay is recommended	Design	—

The Section's quantitative content is limited to the bootstrap simulation under the net_gain model and the comparison of three policies. The saturation of the alternative models is documented as F-016.

The selection criterion is the activity–reputation correlation reported in Section 8: 0.76 for net_gain versus 0.08–0.11 for the saturating models [7], which is the property that matters for the admission rule. The standard deviation of reputation under net_gain is 0.0424 at the sweep optimum $\delta/g = 1.5$ (five seeds), and it is reported for completeness, not as a pass/fail threshold. The sweep fixes the operating point $\delta/g = 1.5$; it does not select the model, and no threshold on R_{std} is used.

9 Evaluation

9.1 Scope

This section evaluates three things: the reproducibility of the numerical claims, the measured on-chain gas costs of the cryptographic primitives, and the stability of the conclusions under parameter variation. Two benchmarks are deferred to future work because they require a live network: the proving time of the recursive option (not used by the adopted design), and the latency of cross-chain message propagation.

9.2 Reproducibility

Every claim in Sections 2 through 8 that is backed by a script is accompanied by it in `verification/`. The scripts are self-contained: they depend only on NumPy and SciPy, and they declare their seeds and sample sizes at the top of each file. Running `pip install -r verification/requirements.txt` followed by the commands listed in `verification/README.md` reproduces every number reported in the paper. The README lists the 38 scripts and their expected runtimes.

The project documents its own falsifications in `docs/audit/numerical-verification.md`. As of this writing the log contains 22 entries (F-001 through F-022), each one a claim that was made, tested, and corrected. The log exists because the numerical verification is the primary defense against errors of the kind that would otherwise propagate through the paper unchallenged.

The scripts run in the following time ranges on a 2020-era desktop (Table 9.1):

Table 9.1: Runtime of the verification scripts on a 2020-era desktop.

Script	Time
`heavy_tail_scaled_clique.py`	∼24 min
`heavy_tail_n5000.py`	∼15 min
`recalibrate_top.py`	∼10 min
`bipartite_power.py`	∼3 min
`max_statistic.py`	∼2 min
`calibration_modularity.py`	∼400 s
`dt_convergence.py`	∼130 s
`mfpt_jump_diffusion.py`	∼90 s
`calibration_heavy_tail.py`	∼60 s
`sim_positivity.py`	∼34 s
`clique_spectra_seeds.py`	∼21 s
`censoring_test.py`	∼17 s
`clique_spectra.py`	∼17 s
`stationary_stats.py`	∼12 s
`r_std_specification.py`	∼5 s
All others	< 5 s each

Total runtime for the full suite: approximately 67 minutes on a single core. The dominant costs are the heavy-tailed recalibrations (`heavy_tail_n5000.py`, ∼15 min; `heavy_tail_scaled_clique.py`, ∼24 min) and the RMT calibration (`recalibrate_top.py`, ∼10 min).

9.3 On-Chain Gas Costs

The cryptographic design of Section 4 has an on-chain component. We measure the gas cost of the EVM precompiles that implement the BN254 pairings and scalar multiplications [`integrations/avalanche/benchmarks/src/GasBenchmarks.sol`]. The measurements match the EIP-1108 revision of EIP-196/197 to the gas unit (Table 9.2):

Table 9.2: Measured gas against the EIP-1108 revision of the EIP-196/EIP-197 formula.

Primitive	Measured gas	EIP formula
`ecAdd` (0x06)	292	150 + overhead
`ecMul` (0x07)	6,142	6,000 + overhead
`ecPairing` $k = 1$	79,142	45,000 + 34,000+ overhead
`ecPairing` $k = 2$	113,142	45,000 + 68,000+ overhead
`ecPairing` $k = 4$	181,142	45,000 + 136,000+ overhead

The overhead is 142 gas in all five measurements, consistent with a single `staticcall` dispatch. Groth16 verification over BN254 uses $k = 4$ pairings and therefore costs 181,142 gas for the pairing step alone, plus approximately 25k gas for the surrounding verification logic. The total with the verifying key inlined is approximately 206k gas, constant in the size of the circuit; with the key in storage the same check costs 223,403 gas warm and 286,144 gas cold, and the cold figure is the one used below.

The V3 Groth16 proof of Section 4 verifies directly at 286,144 gas on the cold path of `AVIPRegistry.verifyThreshold` with V3's three-input key (223,403 warm), measured with the real proof; the four-input key the registry installs measures 306,253 gas cold and 247,211 warm, and that is the deployed figure; the 206k composition assumes an inlined verifying key and is neither. The Bulletproofs alternative for $n = 500$ elements requires $2n$ scalar multiplications and $2n$ point additions, approximately 6.5M gas, or 21× the deployed Groth16 cost; this is why Option B is not adopted. The recursive option is estimated at approximately 250k gas but is not used or measured.

The gas cost and the stake reduction can be compared directly. At the mean validator the bootstrap of Section 8 reduces the stake from $\Theta = 1.0$ to $S_f^* = 0.7764$, a saving of $\Delta S_{\text{mean}} = 0.2236$ units of stake. The break-even gas price is computed over a reference horizon of $k = 10$ epochs, at the upper end $\omega = 0.10$ of the cost-of-capital support, with the Groth16 verification paid once per admission:

$$g^* = \frac{k \cdot \omega \cdot \Delta S_{\text{mean}}}{G} \times 10^{18} \text{ wei per gas}, \tag{9.1}$$

where one unit of stake is 10^{18} wei; if one unit of stake is worth p native units, the threshold scales by p. For the reference mock token, $p = 1$, and the figure scales linearly with p. For $G = 306{,}253$ gas, the cold verification path under the installed key, $g^* \approx 730$ gwei; for the full admission through `manager.admit` — cold, first admission, under the same key — $G = 395{,}255$ gas and $g^* \approx 566$ gwei. The two are not mixed: a comparison of verification alone uses 306,253, a comparison of complete admissions uses 395,255. Under V3's key the verification alone gives 781 gwei, and the V3-era admission measurement of 376,588 gas gives 594 gwei; that measurement is superseded, since it was taken under a key nothing installs and on a manager that has changed since. The Fuji receipts record an effective gas price of 160 wei/gas; at that testnet fee the verification costs 4.90×10^{-11} AVAX, almost ten orders of magnitude below break-even. At a realistic 25 gwei it costs 0.00766 AVAX, still 29 times below break-even. The 160 wei figure is a testnet observation, not a protocol constant. Below-mean validators save less, and their break-even gas price is proportionally lower.

The measurement uses a local Foundry instance with Solc 0.8.33. The precompile gas schedule is a constant of the EVM protocol, so the measurements apply identically to any EVM-compatible chain including Avalanche C-Chain.

9.4 Stability of Conclusions

The conclusions of the paper depend on parameter choices that are fixed at design time. We check that small variations do not change the qualitative outcome.

Reputation dynamics (Section 2). The domain validity of the multiplicative-jump specification is verified pathwise: 0 of 100,000 trajectories leave the positive domain for any parameter choice that satisfies Feller. The mean first-passage time converges to the exact value as $dt \to 0$, with a relative error of 0.22% at $dt = 5 \times 10^{-4}$. The conclusions do not depend on the discretization.

Collusion detection (Section 5). The null quantile $q_{99} = 9.0542$ is calibrated with $N_{H_0} = 2000$ replicates. The standard error of the q_{99} estimate is $\sqrt{p(1-p)/N}/f(q_{99})$, evaluated at the empirical z-score of q_{99} (2.568), giving 0.017 in $\lambda_{\max}(B)$ units, or 0.186% of its value. The 95% confidence interval is approximately $[9.021, 9.087]$. The corresponding theoretical value, using the standard normal density at $z = 2.326$, is 0.103%. The lowest true positive rate across the five simulated configurations is 0.972 (single clique of size 20), well above the $\alpha = 0.01$ threshold. The qualitative boundary does not depend on the calibration sample size.

Data layer (Section 7). The conditional model $n^*(B) = \lfloor B/B^* \rfloor$ is exact given the exogenous parameters. The attacker profit is -502 in the model with slashing $S_{\text{slash}} = 10$ and $+49.91$ without detection; $-502 = -10 \times 50.2$ is the seed-42 mean, and the model value at exactly 50 frauds is -500. Both figures are model bookkeeping under the coverage convention of Section 7, not executable losses: the loss collateralized on-chain is the escrowed stake of Section 6, which is at least $S_f^*(T) \in [0.6, 1.0]$ and unbounded above, because the admission path caps nothing. The $\Theta = 1.0$ that bounds the model is a coverage convention, not a cap the contract enforces. The detection latency enters as a declared constraint: the deterrence condition (7.5) is evaluated over k epochs [`verification/mechanism/m05_attack_model.py`], where the probability the worst-case type requires rises from 0.0625 at one epoch to 0.375 at the nine the freshness bound admits, and the certified norm cannot exceed 1, so those are the worst values over the domain the circuit certifies (the unconstrained limit, 0.2857 and 0.7826, is not reachable). The discontinuity at $B = B^*$ is a property of the conditional model, not of the Monte Carlo. The wall-clock scale of k epochs is $k\, N_b\, B_{\text{blk}}$, where N_b is the number of blocks per epoch and B_{blk} is the block time (distinct from the bounty B). N_b is a deployment parameter and is not fixed on-chain. The measured C-Chain block time over $N = 100$ blocks gives $B_{\text{blk}} = 1.0$ s at the median, 11.0 s at p95, and 28.0 s at the observed maximum; the conservative scale uses the upper tail, not the median.

Bootstrap (Section 8). The stationary reputation under net_gain is $R^* = ga/(ga + \delta)$. The measured mean $R = 0.3936$ matches the analytic $R^* = 0.4$ for unit activity. The choice $\delta/g = 1.5$ maximizes R_{std} in the sweep; nearby values give similar results.

9.5 Deferred Benchmarks

Two benchmarks are not reported in this paper because they require a live network:

Recursive option proving time. The recursive construction of Section 4.4 is not used by the adopted design. Its proving time depends on the recursion framework and the inner circuit; measuring it requires implementing the recursive circuit, which is outside the scope of the current paper.

Teleporter latency. The protocol uses Avalanche Warp Messaging (AWM) through the Teleporter contract to propagate reputation commitments across subnets. The message latency depends on the Avalanche consensus timing and is bounded by the finality time of the source subnet. This is not measurable without a live testnet deployment.

Both benchmarks are declared as future work. The protocols for measuring them are defined in the repository.

The gas of the Fuji deployment is measured and reported in Table 4.4. Fuji C-Chain block time was measured over $N = 100$ blocks (median 1.0 s, p95 (nearest-rank) 11.0 s, min 0.0 s, max 28.0 s). This measures block time, not epoch duration: an epoch is the protocol's commitment interval, and its block count is a design parameter not fixed on-chain. The deterrence condition of Section 7 is expressed in epochs, so a wall-clock conversion requires fixing that parameter; once it is fixed, the conservative scale for k epochs is k times the epoch duration computed from the upper tail (11.0 s per block at p95, 28.0 s at the observed maximum), not from the median. The 100-block sample is a low-traffic testnet observation, not a protocol constant; a deployment should define epochs by timestamp or fix N_b explicitly.

Fuji phase 1 is a deployment and gas test, not an incentive test. It demonstrates contract creation, role wiring, the deployment gas of Table 4.4, the `verifier = address(0)` guard, and the C-Chain block-time sample. It does not demonstrate incentive compatibility, real economic value at risk,

challenge latency, the Teleporter transport, or the real verifier. Teleporter cross-subnet latency remains deferred to Phase 4.

Fuji phase 2 adds three things and no fourth. It re-deploys the registry, the vault and the manager with a 2-of-3 Safe as `slashAuthority` (Table 4.4, 2026-10-23); it repeats the `verifier = address(0)` guard on the new registry; and on 2026-10-24 it deploys a `Groth16Verifier` and announces it through the registry's guardian rotation (Section 10). The announcement is a proposal, not an installation: the registry accepts `executeVerifierChange` only after `CHANGE_DELAY` (172 800 s), and until then `verifier()` still reads zero. The key behind that verifier is the committed development setup of the repository, not a ceremony, so the deployment answers whether the wired admission path can be reached at all, not whether the proof system is sound. It is not reached yet: with no epoch posted, `verifyThreshold` reverts `UnknownEpoch` before it consults the verifier, so what remains missing is a posted commitment and a proof, and installing the verifier is a precondition for that test rather than a substitute for it. The two transactions, their receipts and the reads after them are in `integrations/avalanche/scripts/fuji_20261024/`.

9.6 Summary of the Section

Table 9.3 lists the claims of this section with their status and their source.

Table 9.3: Claims of Section 9, their status, and their source.

Claim	Status	Source
Verification suite reproduces all claims	Verified	`verification/`
22 entries documented	Verified	`docs/audit/`
`ecPairing` $k = 4 = 181{,}142$ gas	Measured	`GasBenchmarks.sol`
Groth16 verification, cold path: 306,253 gas, installed key (286,144 under V3's)	Measured	`docs/audit/evidence/2026-10-28-v3-vs-m4d-gas`
Bulletproofs direct: ∼6.5M gas	Estimated	`GasBenchmarks.sol`
V3 (global norm): 94,438 constraints, 3.9 s	Measured	`src/zk/scripts/m4b_output.txt`
Recursive option: ∼250k gas (not adopted)	Estimated	—
Conclusions stable under parameter variation	Verified	Multiple
Break-even gas price at mean stake reduction	Analytical	Section 9.3
Epoch block count N_b is a deployment parameter	Declared	Section 9.4
Recursive option proving time (not adopted)	Deferred	—
Teleporter latency	Deferred	—

The evaluation confirms the reproducibility of the numerical claims and the measured gas costs of the cryptographic primitives. The two deferred benchmarks are declared as future work with the protocols for measuring them.

10 Limitations

10.1 Scope

This section states the limits of the protocol and of the numerical verification. It complements the methodological limitations of Section 9 (validity of numerical methods, sample sizes, discretization bias) with limits that would persist even if the numerical methods were exact. These are limits of the design itself, not of the simulation.

The distinction matters. If a reviewer asks why the paper does not claim to have solved X, the answer is one of two things: either X was not correctly measured (a limitation of Section 9), or X cannot be solved within the design (a limitation of this section).

10.2 Adversary Model

The protocol assumes a rational, profit-maximizing adversary. Every security argument in the paper is conditional on this assumption. Specifically:

The slashing of Section 7 deters fraud only if the adversary prefers profit to loss. A state actor, an ideological opponent, or a competitor with non-monetary utility is not deterred by slashing. The mechanism design does not apply to them.

The cost of Sybil reputation is a market cost. A validator accumulates reputation by producing consensus events on the source chains, and the cost of these events is set by the market for validation on those chains. A Sybil adversary that controls validators on a source chain pays the marginal cost of those validators, not the market cost. The protocol does not model this case; it assumes the adversary does not control the source chains.

The data layer bounty B^* of Section 7 assumes auditors respond to profit. If no verifier is willing to audit at any bounty, the fraud is not detected. The equilibrium $n^*(B) = \lfloor B/B^* \rfloor$ has a solution only if B is at least B^*. Below that threshold, the mechanism provides no detection regardless of the value of the reputation at stake. It also assumes that a verifier audits on its own account: the bounty is a common pool divided among the auditors who are active, so the income per auditor falls as participation grows, and a verifier that expects the others to audit has an incentive to free-ride on their work instead — a dilution of the individual reward that the model does not represent. The equilibrium table reports the representative auditor's payoff under equal division and full participation, so its verifier-profit column is conditional on full participation, not a second equilibrium.

The fraud rate is endogenous, and Table 7.1 is not an equilibrium in it. The table fixes $p_{\text{fraud}} = 0.10$. If the attacker chooses p_{fraud}, the negative profit at $B \geq B^*$ drives $p_{\text{fraud}} \to 0$, which drives $B^* = c_{\text{audit}}/p_{\text{fraud}} \to \infty$ and $n^* \to 0$; with no auditor, fraud becomes profitable again. Closing that loop requires a mixed-strategy equilibrium of the audit game, which this paper does not solve. The table is therefore comparative statics at an exogenous fraud rate, and the same circularity is the verifier's dilemma named above.

The publisher of a fraudulent leaf is not attributed to a validator in v1. The registry's `postCommitment` is callable only by the sequencer, while the manager's `slash` debits a validator's escrow. If the sequencer publishes the leaf, the validator is not the publisher; if the validator submits it through the data layer, the data layer is a relay. The model's attacker is the publisher, and the executable validator slash requires an attribution path that v1 does not implement. This is a gap in the enforcement chain, not only in the model.

The collusion detector is not connected to the reputation state, and its on-chain flag is not a sanction. Section 5 measures a spectral test on a collaboration graph W that the protocol does not produce: the detector receives W as an argument, T is a different object — one validator's tensor — and no map from consensus events to W is implemented. The on-chain half is incomplete in the same way: `AVIPCollusionAlertChannel` records a flag once per source chain and epoch and reverts with `FlagAlreadyRecorded` on a second alert, with no revocation path, and no deployed contract consumes the flag, so an alert moves no stake and no voting weight. The flag is an alert, not a penalty. Connecting it to one requires, first, the false-positive rate of the test on the graph family actually deployed, which in the Chung–Lu case is 1.0 at $\alpha = 0.01$, and second, a dispute window before any weight is reduced.

The publisher of the commitment is trusted, and the trust is bounded by a delay. The commitment of Section 7 is posted by a single role, and in the delivered registry that role is held by a guardian distinct from the sitting keys: only the guardian may propose a change of the sequencer or of the verifier, the change takes effect only after a fixed delay, and executing a proposed change is permissionless once that delay has elapsed. Both setters emit events, and a proposal of the zero address is rejected. The verifier stays mutable by design: the registry holds the commitment history that the admission rule depends on, so freezing it would turn a circuit defect into a redeployment that loses that history. What this section still assumes is that the guardian does not act against the protocol — a trust assumption about a quorum, not a property of the code.

The detection window of Section 7 is a declared constraint of the model, not an enforced delay. The loss of (7.5) is realized only if the escrow is still posted when the fraud is adjudicated. In v1 that premise holds by construction rather than by enforcement: the manager exposes no exit, and the custody vault caps a withdrawal at the escrow above the stake on record, so an admitted validator cannot withdraw the stake that backs its admission, and the only way out of the admitted set is the slash itself. The gap is therefore not a missing escrow but a missing ordering: an unbonding delay and an on-chain challenge window are follow-up work, not properties of v1, and

an exit path added later would have to be sequenced against the adjudication before (7.5) could be read as covering it.

The epoch duration is a deployment parameter, not an on-chain constant. The deterrence condition (7.5) is expressed in epochs. Converting k epochs to wall-clock time requires the formula $k\,N_b\,B_{\text{blk}}$, with N_b the number of blocks per epoch and B_{blk} the block time (distinct from the bounty B). No contract fixes N_b. The measured C-Chain block time gives $B_{\text{blk}} = 1.0$ s at the median, 11.0 s at p95, and 28.0 s at the observed maximum (Section 9.3); the conservative reading uses the upper tail. The wall-clock bound is therefore conditional on a deployment choice that this paper declares but does not enforce.

The identity layer is assumed, not provided. The non-transferability claim applies to the reputation state, not to the validator's identity. The protocol does not provide an identity layer: it assumes one that makes a validator identity persistent and costly to recreate. Without that layer, selling a signing key would transfer the reputation-weighted income stream even though the reputation token itself cannot be transferred. A secondary market for identities or keys is outside the scope of the model and is declared here as a limitation.

The number in the leaf is attributed by the publisher, not proved. The admission circuit certifies that a leaf Poseidon(i, r) is in the epoch's root and that $0 \le r < 1$; it does not certify that r is the validator's true reputation, because the epoch's entries are written by the sequencer from the ingestion of Section 7 and the circuit never sees the events. The map from consensus events to $T(i)$ is an off-chain computation. Three consequences follow that the protocol does not close on-chain: the epoch may hold the same address more than once with different norms, and the prover then proves the leaf that favours it; the certificate binds an entry, not a validator's tenure, so a validator that acquires a good leaf by any off-chain means is admitted; and the Frobenius norm of the epoch is not what admission prices, so no on-chain quantity measures the reputation of the whole state. Closing the first requires a uniqueness constraint on the epoch and in the circuit; closing the third requires the commitment itself as a circuit input, which is the construction of Section 4 that the delivered circuit does not use.

The contract batch is tested offline, not fully exercised on-chain. M9a–M9c (registry authority, full-stake slash, and ejection surcharge) are implemented and pass the offline test suite. The phase-2 registry was deployed with `verifier = address(0)`; a development-key `Groth16Verifier` was deployed and announced on 2026-10-24, but the registry installs it only after `CHANGE_DELAY`, and no epoch has been posted, so `verifyThreshold` reverts either way — `VerifierNotSet` until the change lands, `UnknownEpoch` after it — and no admission has been certified on chain. A complete on-chain exercise of the admission, slash, and ejection paths therefore remains open work; the offline suite is the evidence for the contract logic in this paper.

These assumptions are standard in the cryptoeconomic literature but they are not universal. The protocol's guarantees do not extend to adversaries that violate them.

10.3 Collusion Detection Boundary

The modularity-spectrum test of Section 5 detects cliques and bipartite structures with high power in the simulated range. The smallest configuration tested (a single clique of size $m = 20$) is detected at 0.972 with $s[-1]$ as the right-tail statistic.

The detection boundary below which the test loses power was not located. Configurations smaller than $m = 20$ were not simulated, and the theoretical Kesten–Stigum threshold for the degree-corrected SBM with the reference parameters ($n = 500$, $\bar{d} = 20$) is below the tested range. The location of the boundary is an open question.

An adaptive adversary who knows the current test may attempt to construct collusion below the (unlocated) boundary. The protocol relies on economic deterrence for this regime rather than on detection.

10.4 Collusion by Bipartite Polarization

The left-tail test measures bipartite polarization via $-\lambda_{\min}(B) > 9.1371$. The power of this test was measured [`verification/rmt/bipartite_power.py`]: true positive rates of 0.990–1.000 across block configurations from balanced (250/250) to heavily unbalanced (50/450), with density ratios $\rho_{\text{between}} \in \{2, 5, 10\}$ and $N_{H_1} = 500$ replicates.

The left-tail test is therefore not a gap. Its power is comparable to the right-tail test. Both tails have been calibrated and measured.

10.5 Reputation Model Selection

The bootstrap analysis of Section 8 uses the net_gain model

$$R' = g \cdot a \cdot (1 - R) - \delta \cdot R \tag{10.1}$$

selected for its correlation between activity and reputation (0.76 vs 0.08–0.11 for the saturating alternatives). This correlation is the selection criterion. The standard deviation of reputation under net_gain is 0.0424 at the operating point $\delta/g = 1.5$ (five seeds; the $N = 100$ mean is 0.043970 with 95% CI $[0.043351, 0.044588]$); it is reported as a sensitivity check, not as a pass/fail threshold.

If the true reputation dynamics differ from net_gain — if the activity of a validator has decreasing returns, if reputation decays with a different functional form, if there are non-linear effects — the bootstrap results change. The reference stake reduction is 22.4% at the mean validator [`verification/bootstrap/simulate_bootstrap_net_gain.py`]. The qualitative ranking of policies (hybrid_no_decay dominates hybrid_decay) is robust to small changes in the gain rule; the specific numbers of other rules are not tabulated in this paper.

10.6 Community Structure and Tucker Compression

Section 3 shows that the reference rank $(4, 3, 3)$ does not capture the blocky test tensor. A tensor constant within K communities in mode C has mode-0 rank at most K; at $K = 3 < R_1 = 4$ the mode-C structure is captured, and the 52.53% error comes from truncating modes D and H to 3. The claim is therefore about this rank, not about block structure in general.

If the real reputation tensor has community structure — if validators cluster by chain, by domain, or by region — the Tucker compression is inefficient for that structure. Whether the real tensor is low-rank or blocky is not resolved in this paper. It is an empirical question that requires multi-chain reputation data.

10.7 Calibration on Synthetic Graphs

The RMT calibration of Section 5 uses Erdős–Rényi and Chung–Lu graphs. These are standard null models but they do not capture all features of real collaboration graphs: community hierarchies, temporal correlations between interactions, and directionality.

The empirical thresholds ($q_{99} = 9.0542$ for the right tail, 9.1371 for the left tail) are valid for the calibration distribution. Their validity on real collaboration graphs is an open question. If the real graphs have different spectral properties, the thresholds need recalibration. The recalibration procedure would be the one described in Section 5.4 (Monte Carlo estimation of the null quantiles), applied to real collaboration graphs instead of synthetic ones.

10.8 Bootstrap Assumptions

The bootstrap simulation of Section 8 assumes:

- Activity follows LogNormal$(0, 0.5)$ per validator.
- Validators enter at a rate of 0.03 per epoch.

- The transition rate $\tau(t)$ is linear in time, reaching 1 at epoch 200.

These are design choices, not empirical facts. The stationary reputation $R^* = ga/(ga+\delta)$ depends on the activity distribution; if real activity is heavier-tailed or lighter-tailed, R^* changes.

The bootstrap numbers (S_f mean 0.7764, R mean 0.3936, median S_f 0.7714, median R 0.4000, Gini 0.1283) are specific to the reference parameters. The qualitative ranking of policies (hybrid_no_decay dominates hybrid_decay) is robust; the specific numbers are not.

10.9 What the Protocol Does Not Claim

For clarity, the protocol does not claim:

- Security against a non-rational adversary.
- A network-level Cost of Attack bound; the paper prices admission, not the cost of acquiring the reputation that earns the discount.
- An equilibrium in p_{fraud}; Table 7.1 is comparative statics at an exogenous fraud rate.
- On-chain attribution of a fraudulent leaf to a validator; v1 does not implement it.
- Trustless binding of the V3 Poseidon root to the keccak Merkle root; the link is off-chain.
- Detection of collusion below the (unlocated) Kesten–Stigum boundary for $s[-1]$.
- Optimality of the Tucker representation for the real reputation tensor.
- Validity of the RMT thresholds on real collaboration graphs.
- Independence of the bootstrap results from the activity distribution.

Each of these is a direction for future work. The protocols for addressing them are defined in the repository.

10.10 Summary of the Section

Table 10.1 lists the limitations with the section that states each.

Table 10.1: Limitations, their category, and the section that states each.

Limitation	Category	Section
Rational adversary assumption	Design	7, 10.2
Sybil cost is market cost	Design	10.2
Bounty threshold B^*	Design	7
Cost of Attack not bounded	Design	6, 10.2
p_{fraud} exogenous; no equilibrium in it	Design	7, 10.2
Fraudulent leaf not attributed to a validator	Design	7, 10.2
Poseidon/Merkle link is off-chain	Design	4, 10.2
Detection boundary not located	Design	5
Bipartite polarization measured	Verified	5
Net_gain does not satisfy all criteria	Selection	8
Tucker fails for the blocky instance at $(4, 3, 3)$	Method	3
RMT calibrated on synthetic graphs	Method	5
Bootstrap assumes LogNormal activity	Method	8

The methodological limitations (calibration on synthetic data, specific model choices) are separated from the design limitations (non-rational adversary, sub-boundary collusion) because the two require different responses. The methodological limits can be addressed by collecting more data and recalibrating. The design limits require different mechanisms or different claims.

11 Conclusion

11.1 Summary

The Diamond Framework replaces a fraction of the financial stake required of a proof-of-stake validator with verifiable, non-transferable reputation. The protocol is built from seven components; six are verified numerically in this paper. The zero-knowledge proof is analysed: the original circuit is shown unsound, and the delivered V3 circuit is measured, although no ceremony-key verifier is deployed (a development-key one is live on the Fuji testnet):

1. **Reputation dynamics** (Section 2). The process is a CIR diffusion with multiplicative downward jumps. It remains in its positive domain by construction. The exact mean first-passage time is computed by numerical integration and confirmed by two independent simulation methods.

2. **Tensor representation and compression** (Section 3). The reputation tensor is compressed by Tucker decomposition for storage and transmission. The compression is exact for low-rank tensors and fails for blocky tensors; it is not used in the security path.

3. **Zero-knowledge threshold proof** (Section 4). The original circuit specification is unsound. The delivered V3 circuit commits to the tensor with a Poseidon sponge and certifies the floor norm in the same statement: 94,438 constraints, 3.9 s proving, 128-byte proof, verified directly with Groth16. Recursion is not used; it is retained only as a deferred benchmark. The EVM precompile costs are measured locally and match the EIP-1108 revision of EIP-196/197 exactly.

4. **Collusion detection** (Section 5). The modularity matrix $B = W - dd^\top/(2m)$ replaces the normalized adjacency matrix A_{sym}. The empirical null quantile at $\alpha = 0.01$ is 9.0542 with

$N_{H_0} = 2000$ replicates. The lowest true positive rate across the five simulated configurations is 0.972; the boundary below which detection degrades is below the tested range. The boundary is characterized as a conjecture.

5. **Payment rule** (Section 6). The payment rule is a saturated uniform posted price, not a Myerson auction.

6. **Data layer** (Section 7). The data layer equilibrium is $n^*(B) = \lfloor B/B^* \rfloor$ with $B^* = c_{\text{audit}}/p_{\text{fraud}}$.

7. **Bootstrap** (Section 8). The bootstrap under the net_gain reputation model reduces the mean financial stake by 22.4%.

11.2 Methodology

Every claim backed by a script is paired with it. Claims that are estimates, design choices, deferred benchmarks, or conjectures state their status. The project maintains a public falsification log (`docs/audit/numerical-verification.md`) with 22 entries: each one a claim that was made during development and tested.

The falsification log includes errors of several kinds: a fabricated simulation figure (F-001), a mismeasurement of a gas cost (F-017), a factor of two in an integral (F-011), a rounding rule that shifted a phase transition (F-014), an unverified projection of a multidimensional type (F-004), and an incorrect eigenvalue statistic in the collusion detection (F-018). The corrections are documented in the same file as the original claims.

11.3 Contributions

The paper contributes:

1. A cryptoeconomic protocol that uses verifiable reputation as partial collateral, with an explicit adversary model and a stated detection boundary.

2. A numerical verification suite that covers every claim backed by a script in the paper, with declared seeds and sample sizes, including local benchmarks of the EVM precompile costs for the cryptographic primitives.

3. A falsification log that documents errors of measurement, analysis, and model selection as they were found, with the corrections.

4. A set of design decisions with explicit boundaries, stating where the protocol cannot deliver a guarantee (Section 10).

11.4 Open Problems

The limitations section (Section 10) and the deferred benchmarks (Section 9.5) identify several open problems that are not resolved in this paper:

- **Non-rational adversaries.** The security arguments assume profit-maximizing adversaries. State actors and ideological opponents are outside the model.

- **Sub-boundary collusion.** The detection boundary below which the modularity-spectrum test loses power was not located. The protocol relies on economic deterrence for this regime.

- **Bipartite polarization.** The left-tail test is calibrated and its power has been measured (0.990–1.000 across tested block configurations).

- **Real reputation data.** The Tucker compression efficiency and the RMT calibration thresholds depend on empirical properties of the reputation tensor and the collaboration graph. Neither is available in this paper.

- **Recursive option.** The recursive construction of Section 4.4 is not used by the adopted design and is not benchmarked.
- **Teleporter latency.** Cross-subnet message propagation requires a live testnet deployment and is not measured.

11.5 Closing Remark

The protocol is in revision. The current document reports the state of the design at the time of writing, with the measurements that have been performed and the claims that remain open. Further development depends on: multi-chain reputation data for calibration, testnet deployment for latency measurement, and external audit for the cryptographic components.

The paper is intended to be readable as a record of what is verified and what is not. The distinction is maintained throughout, and every claim can be traced to a script, a benchmark, or a falsification log entry.

11.6 Summary of the Section

Table 11.1 lists the components with their status and the evidence.

Table 11.1: Components of the protocol, their status, and the evidence in this paper.

Component	Status	Evidence
Reputation dynamics	Verified	`stochastics/`
Tensor compression	Verified	`tensor/`
ZK threshold proof	V3 measured; dev-key verifier on Fuji, no ceremony key	`src/zk/`, `zk/`
Collusion detection	Verified (synthetic ER); heavy-tailed negative	`rmt/`
Payment rule	Verified	`mechanism/`
Data layer	Verified (conditional model)	`data-layer/`
Bootstrap	Verified	`bootstrap/`
Reproducibility	Verified	`verification/README.md`
Falsification log	22 entries	`docs/audit/`
External audit	Not performed	—
Real-data calibration	Open	—
Testnet benchmarks	Open	—

The protocol is ready for external audit of the cryptographic components. The V3 circuit is measured and a development-key verifier runs on the testnet, but no ceremony key is deployed and the recursive option is deferred. The remaining open problems are stated in Section 10 and listed in Section 11.

A Reproduction Protocol

Every claim backed by a script under `verification/` is produced by that script, and the suite is built so that a claim can be re-derived rather than trusted. The chain of custody is closed and it includes this document. `verification/run_all.sh` runs every script and writes the closed list of the outputs it produced to [`verification/output_manifest.txt`]; `verification/fase0_anchor.sh` hashes exactly that list into [`verification/output_manifest.sha256`] and seals it, together with this paper, into [`verification/audit_manifest.sha256`]; and [`verification/anchor_root.sha256`] commits the eleven files of the chain — this paper among them — so that no document of the chain can be edited without invalidating the root. The suite is deterministic: the harness records a byte-for-byte comparison of the outputs of two independent runs in [`verification/fase1_determinism_evidence.txt`], and a re-run reproduces the manifest.

Table A.1 lists the pinned outputs, in the order of the closed list.

Table A.1: Pinned outputs of the verification suite, in the order of the closed list of the manifest. Paths are relative to the baseline directory.

Output
sim_positivity_output.txt
stationary_stats_output.txt
mfpt_bound_explicit_output.txt
dt_convergence_output.txt
censoring_test_output.txt
mfpt_jump_diffusion_output.txt
cr_ide_solver_output.txt
frobenius_counterexample_output.txt
fee_specification_output.txt
fee_specification_net_output.txt
financial_floor_sensitivity_output.txt
m05_attack_model_output.txt
constraint_count_check_output.txt
soundness_zk_check_output.txt
threshold_quantization_output.txt
succinctness_comparison_output.txt
sim_laplacian_output.txt
calibration_modularity_output.txt
calibration_heavy_tail_output.txt
clique_spectra_output.txt
clique_spectra_seeds_output.txt
asym_weights_output.txt
wilson_intervals_output.txt
recalibrate_top_output.txt
max_statistic_output.txt
q99_standard_error_output.txt
bipartite_power_output.txt
heavy_tail_n5000_output.txt
heavy_tail_scaled_clique_output.txt
tucker_analysis_output.txt
simulate_data_layer_v3_output.txt
attacker_profit_analytical_output.txt
compare_reputation_models_output.txt
sweep_net_gain_output.txt
simulate_bootstrap_net_gain_output.txt
r_std_specification_output.txt
read_fuji_gas_output.txt
measure_block_time_output.txt

The reproduction protocol is the same one that produced the numbers in this paper: run the suite, hash the outputs, compare against the committed manifest, and read the paper only through the claims that the manifest pins.

References

[1] Z. Bai and J. W. Silverstein. *Spectral Analysis of Large Dimensional Random Matrices.* Springer Series in Statistics. Springer, 2nd edition, 2010.

[2] F. Bauer and J. Jost. Bipartite and neighborhood graphs and the spectrum of the normalized graph Laplacian. *Communications on Pure and Applied Mathematics*, 2012.

[3] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. Scalable zero knowledge via cycles of elliptic curves. In *Advances in Cryptology – CRYPTO 2014*, LNCS 8617, pages 276–294. Springer, 2014.

[4] S. Bowe, J. Grigg, and D. Hopwood. Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Report 2019/1021, 2019.

[5] F. Chung and L. Lu. The average distances in random graphs with given expected degrees. *Proceedings of the National Academy of Sciences*, 99(25):15879–15882, 2002.

[6] A. Coja-Oghlan. Graph partitioning via adaptive spectral techniques. *Combinatorics, Probability and Computing*, 19(2):227–284, 2010.

[7] L. W. Cong, Y. Li, and N. Wang. Tokenomics: Dynamic adoption and valuation. *The Review of Financial Studies*, 34(3):1105–1155, 2021.

[8] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[9] J. Goodman. Tezos: A self-amending crypto-ledger position paper. Position paper, 2014.

[10] J. Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, pages 305–326. Springer, 2016.

[11] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina. The EigenTrust algorithm for reputation management in P2P networks. In *Proceedings of the 12th International Conference on World Wide Web (WWW)*, pages 640–651, 2003.

[12] A. Kothapalli, S. Setty, and I. Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Advances in Cryptology – CRYPTO 2022*, LNCS 13510, pages 359–388. Springer, 2022.

[13] D. Larimer. Delegated proof-of-stake (DPoS). BitShares whitepaper, 2014.

[14] R. B. Myerson. Optimal auction design. *Mathematics of Operations Research*, 6(1):58–73, 1981.

[15] B. Øksendal and A. Sulem. *Applied Stochastic Control of Jump Diffusions*. Universitext. Springer, Berlin, 2007.

[16] T. Roughgarden. Transaction fee mechanism design. In *Proceedings of the 22nd ACM Conference on Economics and Computation (EC)*, pages 792–792, 2021.

[17] F. Saleh. Blockchain without waste: Proof-of-stake. *The Review of Financial Studies*, 34(3):1156–1190, 2021.

[18] C. A. Tracy and H. Widom. Level-spacing distributions and the Airy kernel. *Communications in Mathematical Physics*, 159(1):151–174, 1994.

[19] L. R. Tucker. Some mathematical notes on three-mode factor analysis. *Psychometrika*, 31(3):279–311, 1966.

[20] E. P. Wigner. On the distribution of the roots of certain symmetric matrices. *Annals of Mathematics*, 67(2):325–327, 1958.

[21] L. Xiong and L. Liu. PeerTrust: Supporting reputation-based trust for peer-to-peer electronic communities. *IEEE Transactions on Knowledge and Data Engineering*, 16(7):843–857, 2004.